OPENBOT / MVP ACCEPTANCE & REPAIR

macOS MVP 测试与修复手册

针对实际仓库的严格验收、问题修复与逐项提交推送规程

执行基线 1.0 • 2026 年 11 月 5 日

100 项验收用例	16 项初始线索	7 份证据模板
全部 NOT_RUN	待当前版本复现	无虚构执行结果

给使用者

在你的实际Mac工作副本中，由Codex或Claude Code按本手册执行。每个独立问题先复现、最小修复与回归，再创建一个原子commit、立即push并核对远端。同一候选的真实系统、外部服务与CI证据齐全后，才判断macOS MVP通过。

阅读顺序

先看范围、真实命令与问题队列；再执行E0–E7阶段；按G00–G15门禁汇总。完整100项操作卡与机器可读JSON、提示词和模板随执行包提供，本PDF保留操作规程与全部用例索引。

已审查的代码快照

xiongcp/OpenBot • main
a223d36a659b7ecf32fca2b4890fc187d3e96e48

本轮完成源码与既有CI只读核对；未在用户Mac执行产品测试，未修改、提交或推送OpenBot代码。本机HEAD若不同，执行前重新绑定。

目录

01 验收目标、仓库基线与证据规则 3

02 macOS环境准备与真实命令 5

03 仓库问题优先修复队列 8

04 严格MVP验收的执行顺序 13

05 真实任务质量、失败定位与证据样例 16

06 OpenBot 严格 MVP 核心验收操作手册 18

07 OpenBot macOS 与真实集成 MVP 验收手册 30

08 OpenBot：逐问题修复、提交与推送规程 43

09 OpenBot macOS MVP：证据驱动的发布门禁 51

10 可直接交给本机 Codex / Claude Code 的执行提示词 56

11 100项用例索引与执行登记 60

12 来源、包内容与验证边界 64

本PDF：流程、诊断、源依据与用例索引。执行包：全部100项详细操作卡、JSON清单、七份证据模板及完整Codex提示词。

01 验收目标、仓库基线与证据规则

版本1.0；日期2026年11月5日；用户时区Asia/Shanghai。

本轮要得到的结果

在用户的Mac上，从真实安装入口启动OpenBot，使用独立测试profile和合成工作区，验证CLI、持久化、审批、受限执行、恢复、飞书、Browser、MCP与两个Provider的实际闭环。每个问题都应留下复现、修复、回归、独立commit、push确认及对应CI记录。最终结论只覆盖实际测试的操作系统、架构、资产与配置组合。

本手册与机器用例清单用于交给Mac本机的Codex执行。本次已完成GitHub源码与既有CI记录的只读审查，没有连接用户的Mac，没有运行该Mac上的测试，也没有修改或推送产品代码。下述源码发现不是本次新执行的运行测试。

已核对的实际仓库

项目	只读核对结果
仓库	https://github.com/xiongcp/OpenBot
分支与快照	main；a223d36a659b7ecf32fca2b4890fc187d3e96e48
快照提交时间	2026-11-05 09:35:26，Asia/Shanghai
Python合同	pyproject要求 >=3.12,<3.13；.python-version为3.12.12
本地自检入口	uv sync --locked；.venv/bin/python scripts/verify.py
可选SDK依赖	provider-openai依赖组；默认只启用dev-contracts
当前任务字段	96项：DONE 4、VERIFIED 12、IN_PROGRESS 67、BLOCKED 3、TODO 10
同SHA的CI	contracts，run 41949551726，结论failure

任务字段可能尚未同步，不能仅凭字段断定代码不存在。验收应以真实仓库、独立执行路径和可复查证据为准。用户Mac上的未推送改动、新分支和后续提交不在此快照内；执行前必须重新记录实际HEAD和工作区状态。

参考来源：仓库README、pyproject、任务状态、CI运行。

当前最需要先解决的事实

1. 真实CLI的main仅暴露version和health；task/approval/reconcile存在于接受注入session的内部函数中。先做真实console脚本测试和入口装配，不能用函数调用代替产品入口通过。
2. 部分macOS验收使用内存Keychain桩、手填隔离声明和manifest文件检查；需要明确标为模拟合同，并补真实OS测试。
3. 崩溃与浸泡harness存在固定效果计数、直接追加通过标签和未杀进程的恢复模拟；必须使用进程外控制器与独立效果账本补证据。
4. 同SHA的CI已经失败：Mac/Windows无法取得指定的3.12.12构建；Ubuntu出现24个测试收集错误，主要为No module named tests，另有2个SDK模块跳过。先修复环境与收集，不能把这些失败归为业务测试通过或单纯缺账号。
5. 某些工作流放在.github/workflows的子目录；官方不支持这种工作流布局。现有macOS工作流还缺少显式依赖准备。

以上发现的复现、修复和回归详见“仓库问题优先修复队列”。这不是对全部源码逐行审计；发现某条路径使用模拟对象，不代表其他未读路径也必然未实现。验收者应追踪产品实际装配图后确认。

先冻结发布范围

资格名称	必需内容	可以声明的范围
Mac开发回归	类型/格式、合同、SQLite、合成HTTP/进程与安全反例	当前源码与受控环境的工程回归
macOS MVP内测	真实Mac进程入口、持久化、Keychain、实际隔离、真实飞书、真实Browser/MCP、云与本地Provider、恢复与产物	记录的macOS版本和架构组合
原设计三平台M3	macOS、Windows、Linux各自通过对应原生门禁，M3功能全量满足	原设计的三平台MVP
桌面分发/M4	Tauri、签名、公证、安装升级及回滚按实际分发形态验收	已验证的桌面发行包

当前先完成macOS MVP内测，不把Windows/Linux缺环境写成Mac测试失败；同时保持原三平台M3未满足的状态。M4签名UI不是源代码方式Mac内测的倒置前提；如果准备向他人分发安装包，则必须补相应安装与签名门禁。Apple Silicon、Intel和Rosetta分别记录，不借用其中一项结果代表另外两项。

什么证据能支持通过

每条用例记录输入、前置条件、实际动作、断言、观察证据、源代码SHA、测试命令、退出码、时间和环境。用例状态采用NOT_RUN、PASS、FAIL、BLOCKED、NOT_APPLICABLE；BLOCKED与NOT_APPLICABLE必须写具体原因、适用范围和下一条件。pytest的skip与xfail不能直接映射为必需门禁PASS。

证据分为四层：源码审查；确定性模拟合同；真实本地进程/SDK/OS；真实外部服务。调用方传入的verified=True、固定计数、文件名含native/e2e、测试标签数量或模型自述，均不足以证明更高一层的能力。报告应显示evidence_kind和未测边界，而不是把模拟合同累计成原生能力。

推荐的三方观测是：产品只读状态、产品审计日志、独立于产品的真实效果记录。控制器记录request_count和effect_count两个不同指标，不能把收到一次HTTP请求当成发生一次业务效果。最终成功状态必须与产物或外部回执一致。

缺陷严重度和测试优先级

分类	定义	处理
S0缺陷	越权副作用、秘密泄漏、不可恢复数据损坏、未知结果导致重复危险效果	停止相关派发；保全证据；优先修复；阻止发布
S1缺陷	关键流程不可用、恢复或隔离失效、验收错误声称通过	阻止相关MVP范围发布，先复现再修复
S2缺陷	有可靠替代路径的非核心功能或性能问题	明确影响、负责人和目标；按发布范围决策
S3缺陷	不改变行为的文案、易用性与低影响问题	可排后续，保留问题记录

P0/P1/P2是测试执行优先级，与S0/S1/S2/S3缺陷严重度分开。P0必需用例不允许通过降低断言或删除测试而变绿。最终已选择发布范围内的所有必需用例均需PASS，缺环境的必需项仍阻塞资格。

原始证据与Git摘要

原始日志、模型响应、测试数据库、截图、诊断包和含租户标识的网络记录放在受限的本地运行目录。入Git的内容限于脱敏摘要、复现测试、合成夹具和必要配置。不要提交Keychain导出、API token、cookie、真实消息正文、完整用户路径或生产数据库。报告必须能指向私有证据的位置或摘要，而不把私有原件复制进公开仓库。

用户已授权的提交规则

用户已明确要求每修复一项问题就提交并推送一次；本机执行代理按已确认仓库和目标分支连续执行，无需每项重复确认。此规则不授权强推、覆盖用户未提交改动、自动合并main、生产部署或向未知真实收件人发送测试消息。真实服务测试限定用户明确配置的专用测试应用和合成收件范围。具体原子提交与推送恢复流程见后续章节。

02 macOS环境准备与真实命令

步骤1：冻结源码，保护正在开发的工作区

先在真实OpenBot仓库根目录读取AGENTS.md、CLAUDE.md、README、pyproject.toml、uv.lock、现有CI及最近提交。执行以下只读检查，记录输出的脱敏摘要：

```
git status --short
git branch --show-current
git rev-parse --show-toplevel
git rev-parse HEAD
git diff --stat
git diff --cached --stat
git remote get-url --push origin
```

远端若含凭据不得进入报告；核对其归属为xiongcp/OpenBot，SSH和HTTPS形式可不同。不要运行git reset --hard、git clean -fd或自动stash用户改动。工作区不干净时，先识别每个改动的归属；需要隔离则从明确的基线新建专用worktree，不把未提交实现丢在原目录后宣称新worktree代表完整产品。

默认在独立mvp/macos-hardening分支执行逐项修复。已有同名分支先检查状态并复用正确进度，不能用-B重置。远端main可能继续前进，记录本轮base_sha；合并更新后需要对新的候选重新做相关回归。

步骤2：记录本机，而不是推断架构

```
sw_vers
uname -m
git --version
uv --version
xcode-select -p
```

若xcode-select不可用，只有依赖原生构建或签名的步骤受影响。记录系统版本、硬件架构、Python架构、是否经Rosetta、容器后端版本和锁定镜像；不记录序列号、私人网络拓扑或全量环境变量。不要用完整env命令收集凭据。

此仓库快照指定Python3.12.12，并要求>=3.12,<3.13。优先复用已经验证的本机解释器；若需安装，先核对下载源和目标架构。uv的可下载Python集合随uv版本固定，缺构建时应把工具链变更作为独立问题处理，保留确切版本与来源，不静默升级到3.13或使用浮动latest。[T04]

步骤3：隔离数据、凭据和证据

为本轮创建独立测试profile、工作区、数据库、Artifact根、浏览器profile、测试Keychain条目名、容器标签前缀与合成外部收件范围。它们应由程序真实配置进入产品，不通过修改HOME影响其他应用。

以下只是创建私有运行目录的通用Mac shell步骤，不会启动产品：

```
umask 077
OPENBOT_QA_RUN_DIR="$(mktemp -d "${TMPDIR:-/tmp}/openbot-mvp.XXXXXX")"
printf '%s\n' "$OPENBOT_QA_RUN_DIR"
```

将此绝对路径保存到本地运行记录。目录尚未加入Git，原始结果均写在此处。证据按run_id分目录，禁止覆盖上轮成功或失败记录。每个运行记录至少含source_sha、tracked_tree_sha、dirty状态、lockfile_sha256、系统/架构、Python版本、命令argv、退出码、开始结束时间、用例ID与证据摘要。

scripts/verify.py目前会写入仓库的evidence/implementation并覆盖固定文件名。初次使用应在专用worktree运行，将结果归档到私有run目录并辨认生成文件；不要把它们无差别加入修复commit。计划中的--output-dir等参数尚未实现，不能直接当成现有命令使用。

步骤4：恢复锁定环境与测试收集

在真实仓库根目录逐条执行；每条保留退出码，失败先分类：

```
uv sync --locked
.venv/bin/python -c 'import sys, platform, sqlite3; print(sys.version); print(platform.system(), platform.machine());
print(sqlite3.sqlite_version)'
.venv/bin/python -m pytest --collect-only -q
```

收集阶段不允许出现ImportError、未知marker、重复模块冲突或误用其他仓库的tests包。当前CI使用pytest可执行入口，而本地verify使用python -m pytest；后者会把当前目录加入sys.path，这是已观察到No module named tests的优先排查点。[T03] 候选修复为统一从仓库根运行python -m pytest并检查实际收集清单；不要先删除失败测试或把tests目录排除。

若本机editable安装问题导致openbot无法导入，临时PYTHONPATH只作为定位手段。正式安装与分发门禁必须在没有此补丁环境的情况下通过；不能把“源码路径可导入”报告成安装包可用。

步骤5：先跑工程回归，再跑SDK与原生组件

当前快照中确实存在的自检命令：

```
.venv/bin/python scripts/verify.py
.venv/bin/python -m ruff check
.venv/bin/python -m ruff format --check
.venv/bin/python -m mypy
.venv/bin/python -m pytest -q -ra --junitxml="$OPENBOT_QA_RUN_DIR/baseline.xml"
```

verify脚本只接受--phase M0或M1，默认M0，并主动清除OPENBOT_NATIVE_TESTS。它不是完整M3原生验收入口；不要虚构--phase M3或将脚本输出PASS等同macOS MVP通过。脚本当前每条子命令超时60秒，TimeoutExpired也需记录为本次运行失败/超时，不能读取上轮残留报告判绿。

为实际OpenAI SDK对合成loopback服务的合同测试，使用已存在的依赖组：

```
uv sync --locked --group provider-openai
.venv/bin/python -m pytest tests/components/contracts -q -rs --junitxml="$OPENBOT_QA_RUN_DIR/sdk-loopback.xml"
.venv/bin/python scripts/verify.py --phase M1
```

这一步证明SDK在受控HTTP错误、重试、取消等场景中的行为，不证明真实云账户、模型质量或实际计费。必须检查收集与skip结果，不能让importorskip隐藏必需SDK用例。后续uv sync若未包含该组可能改变当前环境，重复运行时记录准确

命令。

原生Docker测试应先阅读tests/components/native_poc/docker_poc.py。该快照硬编码开发者机器的Docker二进制与个人socket，并对合成输出目录使用宽权限；必须先按MVP-016修复可移植配置与最小权限，再核对固定镜像已预置、资源配额、测试标签、网络关闭和清理归属。满足条件后，仅运行已有原生用例：

```
OPENBOT_NATIVE_TESTS=1 .venv/bin/python -m pytest \
 tests/components/native_poc/test_docker.py::test_actual_native_container_limits_and_independent_lifecycle \
 -q -rs --junitxml="$OPENBOT_QA_RUN_DIR/docker-native.xml"
```

不能通过verify.py打开该项，因为verify会清除开关。容器测试只允许清理本轮登记的资源，不使用docker system prune。Docker Desktop中的Linux容器资格与macOS宿主进程、Keychain、登录会话资格分开记录。

步骤6：检查产品真实入口是否接通

```
uv run --no-sync openbot --help
uv run --no-sync openbot version
uv run --no-sync openbot health
uv run --no-sync openbot task --help
```

在审查快照中，main仅接受version和health；最后一条预期暴露入口缺口。health在未注入session时会返回daemon_running=false和CAPABILITY_UNAVAILABLE，即使退出码为0也不能代表daemon健康。

先修复真实CLI与本地认证API、daemon发现和凭据会话装配，再执行task创建、审批、取消、恢复的端到端测试。入口修复必须通过全新子进程调用已安装console脚本，不允许测试直接传入CliApiSession。当前没有核验到可用的openbot start、doctor或profile参数，因此本手册不编造启动命令；执行代理须找到并记录真实入口，缺失则作为产品问题实现。

启动后验证监听仅在约定的loopback/本地IPC上、无错误用户实例、无匿名敏感API、启动失败有明确状态。认证token不得放在命令参数、普通日志或README。真实通信载体不能由_InProcessTransport代替。

步骤7：模块回归命令映射

关注模块	当前仓库存在的测试路径	证明边界
合同与协议	tests/contracts、tests/protocol	规范化、模型协议及部分确定性恢复
本地API	tests/contracts/local_api、tests/security/local_api	需区分直接handle与真实socket入口
SQLite与恢复	tests/integration/storage	参考真实SQLite路径；不能自动证明物理断电
CLI	tests/integration/cli	需增加installed console子进程路径
飞书	tests/integration/feishu	受控harness与真实租户分别记录
Browser/MCP	tests/integration/browser、tests/integration/mcp	现有fixture不替代真实浏览器与stdio进程
macOS	tests/integration/platform/macos	当前含桩/声明；修复后再作为native gate
浸泡	tests/chaos	当前并非真正长时长、真实断网或满盘验收

调用形式统一为.venv/bin/python -m pytest 后跟具体路径与-q -ra。仅在修改实际影响对应边界时扩大回归，不为了“严格”机械重跑所有高成本真实服务。最终候选必须统一跑完已冻结的全部必需门禁。

03 仓库问题优先修复队列

使用规则

本队列基于main快照a223d36a659b7ecf32fca2b4890fc187d3e96e48和该SHA的既有CI日志。MVP-001/002有CI观察证据；其他项主要是源码观察与覆盖缺口，须在用户Mac上做最小复现并追踪实际装配路径。全部初始OPEN，未执行本地修复或push。

修复顺序建议为基础CI与收集、验收真实性、产品入口和运行服务、真实能力、最终门禁与文档。可以先针对同一问题编写会失败的回归用例，但不要把多个独立根因合成一个“大修复commit”。修完一项、通过对应本地回归后，提交并确认推送，再开始下一项。

这些16项是待分诊的问题线索，不预设最终恰好16个修复commit。若同一线索包含多个独立根因，拆成明确子问题并逐项提交；若另一条已被同一根因修复，关联证据即可，不制造重复提交。

同一候选尚有其他已知CI基线错误时，前一修复可处于PUSH_CONFIRMED / CI_BLOCKED，不能标FIXED。允许继续修已独立定位的其他阻塞；最终在包含这些修复的候选SHA上统一复验。详见逐问题提交章节。

MVP-001：Mac和Windows的CI解释器准备失败

证据等级：CI_OBSERVED。建议严重度S1、优先级P0。contracts运行41949551726中，Mac arm64与Windows x64在setup-python步骤无法获取3.12.12；后续测试步骤全部跳过。这不是这两个平台的应用测试失败，也不是通过。

复现步骤：读取相同job日志；在拟使用的runner/架构上查询可用Python构建；用相同锁与解释器请求重新准备环境，记录下载来源与退出状态。

修复方向：先验证保留3.12.12并改用uv managed Python是否可行；若固定uv版本没有该构建，单独验证新的锁定工具版本。确需更换Python补丁版本时，在>=3.12,<3.13内选定各目标架构实际可获取的版本，统一.python-version、CI及环境证据。禁止仅删掉版本约束或升级3.13来绕过。

回归：新的Mac/Windows CI必须实际打印Python版本与架构、完成uv sync --locked并进入测试收集。报告分开记录“环境启动成功”和“应用测试通过”。建议commit主题：fix(ci): resolve locked Python bootstrap on target platforms。

MVP-002：CI测试收集出现No module named tests

证据等级：CI_OBSERVED。建议S1/P0。Ubuntu同SHA的pytest退出码2，报告24个收集错误及2个SDK模块skip，失败集中于tests.*跨模块导入。

复现步骤：在干净checkout从仓库根分别执行uv run --no-sync pytest --collect-only -q和uv run --no-sync python -m pytest --collect-only -q，保留两份日志、收集数量和模块来源；核对tests/__init__.py和sys.path。第二种入口成功也不能掩盖当前CI原入口失败。

修复方向：优先统一CI与本地验证的python -m pytest入口，并确认仓库根及源码路径的明确设置。[T03] 如仍失败，检查editable安装、包名冲突和导入模式。共享测试夹具可后续迁到tests/support或conftest，但不为此进行无关重构。不要跳过tests/integration或吞掉ImportError。

回归：相同CI入口收集数达到冻结清单，0 collection error；实际运行关键storage、CLI、API测试。建议commit主题：fix(test): make repository test imports consistent in CI。

MVP-003：工作流存放在不受支持的子目录

证据等级：SOURCE_OBSERVED。建议S1/P0。当前nightly/soak.yml、pull_request/gate.yml、release/supply_chain.yml位于.github/workflows子目录。GitHub官方说明不支持workflows子目录。[T01]

复现步骤：核对精确目录和触发事件，在GitHub Actions中查找对应工作流及最近运行；不能只用文件存在作为执行证据。

修复方向：将各工作流移动到.github/workflows根目录，采用不冲突文件名；同步引用这些路径的测试和文档，保持原触发目的。定时任务需要满足默认分支与调度条件，修复分支先用允许的手动或push触发验证，不自动合并main只为触发schedule。

回归：对新SHA观察到工作流被识别、触发并运行实际步骤；检查没有把旧文件和新文件同时变成重复执行路径。建议commit主题：fix(ci): register acceptance workflows at supported paths。

MVP-004：macOS专项工作流没有准备锁定环境

证据等级：SOURCE_OBSERVED。建议S1/P0。platform-macos.yml只有checkout与python3 -m pytest，没有明确设置Python、安装锁定依赖或报告版本；文件中的job名称不能证明原生能力。

复现步骤：在干净runner上执行现有workflow，检查pytest与所需包从何处获得。缓存/预装环境的偶然成功不作为干净安装证据。

修复方向：加入锁定解释器与依赖准备；把确定性合同、实际SDK、需要Docker或交互用户会话的原生测试分清。托管Mac runner若无法提供产品所需后端，标具体门禁BLOCKED，安排用户Mac或专用runner，不伪造原生通过。

回归：日志记录OS/架构/锁hash/依赖组；失去必要依赖应明确失败或阻塞；所选必需用例不可全部skip。建议commit主题：fix(ci): bootstrap macOS acceptance dependencies explicitly。

MVP-005：macOS 14托管runner标签已经退役

证据等级：SOURCE_OBSERVED及官方公告。建议S1/P0。仓库多处使用macos-14；GitHub公布所有macOS14镜像于2026年11月2日退出GitHub Actions。[T02] 本次CI日志的直接错误仍是Python构建获取失败，不能把两种根因混为同一个。

修复方向：选择当前支持且明确架构的runner标签，例如核验后的macos-15或macos-26，记录实际uname和系统版本。托管runner升级不等于改变产品最低支持的macOS版本；如果仍承诺macOS14用户兼容，应在相应原生设备保留兼容验收。

回归：新标签被调度，架构与期望一致，完整依赖准备和对应测试实际执行。建议commit主题：fix(ci): migrate retired macOS runner labels。

MVP-006：真实CLI没有接通task和approval命令

证据等级：SOURCE_OBSERVED。建议S1/P0。services/core/src/openbot/cli.py的main只接受version和health，内部run_cli_command支持更多动作但需要注入CliApiSession。文档提供的task create等真实命令与console入口不一致。

复现步骤：在干净安装中使用子进程运行openbot task --help，再在独立测试profile验证task创建。保留退出码、stdout/stderr和daemon是否收到真实请求；不要直接调用run_cli_command(session=fake)。

修复方向：共用命令解析器；装配实际loopback/IPC transport、profile发现和受保护认证会话；实现不可用daemon的清晰错误。不得为了让命令工作绕过Local API、直接写SQLite或将token塞进argv。

回归：installed console脚本在新进程完成create/show/approval/cancel/reconcile；错误token、跨profile、daemon停止、响应丢失均有负向结果。health必须检查实际daemon字段，不能只看退出码0。建议commit主题：fix(cli): connect authenticated product command entrypoints。

MVP-007：用户服务安装器只生成JSON描述

证据等级：SOURCE_OBSERVED。建议S1/P0。application/daemon_installer.py当前仅写openbot-user-daemon.json，未在该实现中执行launchd注册或提供真实服务生命周期。文件存在不足以证明用户服务安装。

复现步骤：安装测试profile后，在本机用户会话查询对应服务，观察进程PID、父进程、实际监听、日志、停止和重启。登记的label应从真实配置读取，不硬编码猜测。

修复方向：补真实用户级服务与受控daemon入口的装配，或者明确将当前函数作为manifest生成器，并由独立已实现的installer负责真实注册。不得使用root系统服务替代当前用户会话；工作区与数据不能随卸载删除。

回归：登录会话内start/status/stop/restart、第二实例、关闭终端、注销/重新登录按产品声明验收；崩溃重启不放大授权。M4的Tauri窗口测试仅在UI实现后执行。建议commit主题：fix(platform): wire macOS user daemon lifecycle。

MVP-008：macOS门禁把模拟证据标为原生通过

证据等级：SOURCE_OBSERVED。建议S1/P0。macos_gate.py中_KeychainStub使用内存字典；文件测试只读写a.txt；daemon检查manifest存在；Browser使用手填隔离声明和内存lease。返回字段却包含keychain_compartment_verified和filesystem_and_symlink_verified。

复现步骤：使真实Keychain后端不可用、不给启动浏览器能力、使用不支持的OS环境运行现有harness，观察其是否仍给出对应verified。这里只是在隔离测试里替换能力，不破坏用户真实Keychain或系统设置。

修复方向：保留有价值的模拟合同测试并标SYNTHETIC；将真实OS门禁改为实际调用、独立观测和环境条件。Keychain应创建专用canary、读回/轮换/撤销并清理；文件门禁应覆盖真实symlink/大小写/Unicode路径。缺能力输出BLOCKED，不能仅把失败吞成False后继续标整体PASS。

回归：真实能力缺失时该资格不能通过；真实canary和文件反例成功才产出证据记录。建议commit主题：fix(qa): separate synthetic macOS checks from native qualification。

MVP-009：崩溃与浸泡结果含未执行的通过标签

证据等级：SOURCE_OBSERVED。建议S1/P0。chaos_soak.py直接追加sqlite_backup_and_disk_full_guard；dangerous_blind_retry_count固定0。vertical_slice的故障路径未执行真实SIGKILL/进程重启，效果数由变量赋值；部分核对证据也是harness自行生成。

复现步骤：用独立控制器记录产品PID及启动代次、外部效果账本和数据库重开事件；确认现有测试是否真的制造各标称故障。将效果观察器置于故障进程外，禁止由待测报告自己证明效果次数。

修复方向：加入确定性故障屏障，在已知事务提交点由控制器终止目标进程并等待退出；以同一持久profile重启产品，读取真实WAL/状态与效果账本。满盘使用受限临时卷或配额，不填满用户系统盘。睡眠与断网分别增加真实Mac场景，时钟模型测试继续保留。

回归：覆盖claim前、claim后dispatch前、dispatch提交后发包前、效果后回执前、持久化成功后反馈前；UNKNOWN不得自动重发。任何统计字段来自记录事件而不是固定常量。建议commit主题：fix(qa): measure real crash recovery and external effects。

MVP-010：Browser验收未证明真实启动和跨进程profile互斥

证据等级：SOURCE_OBSERVED。建议S1/P0。抽查的BrowserProfileSessionManager仅用内存字典维护lease并接受隔离布尔声明；该文件没有启动Playwright。它可作为合同层，但不能单独证明浏览器原生能力。

复现步骤：从两个独立进程竞争同一测试profile；启动实际锁定Chromium并导航到受控页面；观察真实PID、profile文件、出口和回收。追踪其他启动适配器是否实际装配，避免仅据单文件断言整个项目缺实现。

修复方向：把已实现的真实启动适配器接入产品；缺失时实现明确的浏览器worker与OS级profile归属。Chromium自身沙箱和外层文件/网络边界分别验收；Service Worker、iframe、重定向、下载与弹窗一起验证。

回归：跨进程争用只有一个成功，未授权出口无实际连接；进程崩溃后不靠删除锁文件强行接管旧活动实例。禁止no-sandbox降级。建议commit主题：fix(browser): qualify real workers and cross-process profile ownership。

MVP-011：MCP验收没有实际stdio握手和流式输出上限

证据等级：SOURCE_OBSERVED。建议S1/P0。抽查的negotiate_initialize只比较传入版本字符串；bound_response_bytes接收完整bytes后截断。这证明部分合同逻辑，不证明真实进程、JSON-RPC握手和读入期间的内存上限。

复现步骤：启动固定包的测试MCP server，记录真实initialize/initialized、tools/list与tools/call；注入不匹配版本、半帧、无限流、stderr洪泛、超时和成功后断连。效果账本独立记录tool是否真的执行。

修复方向：接通或实现受限stdio进程与协议适配器，按流读入预算与帧边界拒绝过量输出，清理归属与deadline可验证。工具定义hash变化触发重新授权，server不能通过自己声称readOnly获得权限。

回归：握手失败不能返回协商成功；内存上限在读入期间生效；未知效果不自动新建ID重试。建议commit主题：fix(mcp): exercise bounded stdio transport and real protocol handshake。

MVP-012：组合MVP用模拟Ollama代表云与本地两Provider

证据等级：SOURCE_OBSERVED。建议S1/P0。mvp_regression.py给_OfflineOllamaTransport固定响应，手填runner_offline_egress_verified=True，并把这一段计为cloud_and_local_providers；该段没有调用真实云Provider。

修复方向：将报告拆成模拟Provider合同、实际SDK-loopback、真实云服务、真实本地runner四个资格。真实云使用受限测试账户与预算；真实Ollama记录模型权重、runner配置和整个进程树出口。缺少其中一项不能用另一项的PASS代替。

回归：不给云凭据时真实云资格BLOCKED；停止本地runner时本地资格失败且无云回退；真实工具计划仍由Broker执行。任务完成率、token和成本来自实际响应与记录。建议commit主题：fix(qa): qualify cloud and local providers independently。

MVP-013：Mac试点发布判定没有纳入全部必需证据

证据等级：SOURCE_OBSERVED。建议S1/P0。mvp_release_gate.py对供应链缺失加入blocking_reasons，但pilot_ok仅依赖security_critical_failures和darwin_arm64_verified。手填布尔值也不足以证明当前候选实际通过。

复现步骤：传入Mac已验证但供应链记录缺失/资产hash不符的报告，观察pilot标志；再使用不同SHA的过期通过记录，验证候选是否被错误接受。

修复方向：冻结Mac内测与三平台发行各自必需门禁；由受信验收记录汇总，校验source_sha、tree/lock/资产hash、平台与evidence_kind，不能只判断artifact数大于0。Mac范围不用Windows/Linux作为前提，但本范围供应链、真实执行和关键安全门禁必须满足。

回归：逐项删除、篡改或跨SHA替换必需证据都阻止相应发布资格；全部当前证据成立才通过。建议commit主题：fix(release): bind macOS readiness to required candidate evidence。

MVP-014：验证记录可能覆盖历史或残留旧通过结果

证据等级：SOURCE_OBSERVED。建议S1/P0。verify.py把日志写到固定路径，子命令60秒超时后会抛异常而未必生成本轮完整失败报告；报告也没有绑定source SHA和运行ID。

修复方向：为每轮生成独立run_id与输出目录，记录每条命令的起止时间、退出码、timeout、日志hash与未运行项目；超时和异常也生成本轮终态报告并非零退出。启动时创建IN_PROGRESS记录，完成后原子替换为COMPLETE；禁止消费者把缺失新报告解释为上轮PASS。不要简单取消所有超时，分别设置短回归与长时浸泡预算。

回归：注入子命令失败、超时、写报告失败与控制器中断，均不得留下可误读的本轮PASS；历史证据不被覆盖。建议commit主题：fix(qa): make verification evidence run-scoped and fail closed。

MVP-015：文档、任务状态与真实资格存在冲突

证据等级：SOURCE_OBSERVED。建议S2/P1；若影响发布声明则提升S1。README新进度、getting-started、external-environments和部分AGENTS段落反映了不同阶段，部分仍写M1未完成或全部TODO；本次任务字段也并非全部DONE。

修复方向：以实际完成的用例与证据为依据生成能力矩阵，分开implementation_status与verification_status；明确旧证据的日期、适用SHA和历史性质。不要批量把96任务改DONE或用更新文案替代产品接线。

回归：文档示例从真实console可执行；能力矩阵每项指向当前证据；未测/缺环境与M4范围清晰。建议commit主题：docs(qa): reconcile implementation claims with verified capabilities。

MVP-016：Docker原生PoC绑定另一台开发机器

证据等级：SOURCE_OBSERVED。建议S1/P0。tests/components/native_poc/docker_poc.py硬编码Docker二进制绝对路径和个人用户socket，并对临时输出目录设置0o777。该文件自述为合成PoC，不是生产executor；因此不能据此判断生产隔离一定存在同样问题，也不能在其他Mac上直接把运行失败当产品失败。

复现步骤：检查当前Mac真实Docker CLI位置、当前用户受控engine/socket及固定镜像是否存在；用受限测试配置记录能力检测结果，确认脚本没有连到意外远端context。不要先改成系统root socket或提权执行来通过。

修复方向：从显式测试配置选择并验证CLI与engine，避免隐式继承未知DOCKER_HOST/context。输出使用预先设计的最小写权限，优先容器内部产物在停止后受限导出并验证，再写入私有宿主目录；不复制宽权限作为通用修复。记录每个已创建容器的ID和本轮标签，清理只作用于这些资源。

回归：两种合法本机CLI/socket布局均可定位；不存在、权限不足或指向非允许engine时明确BLOCKED/CAPABILITY_UNAVAILABLE；运行后的真实限制与stop/wait/export/remove检查通过，所有本轮资源被核对。建议commit主题：fix(test): make native Docker qualification portable and scoped。

来源：docker_poc.py。

修复后还必须执行的完整链路

上述问题修复不能代替后续用例：真实飞书发起任务、配对、审批与反馈；真实浏览器和MCP效果核对；真实Mac休眠、重连、用户会话和容器回收；旧数据恢复；发布资产首次离线使用；模型任务质量和预算。每个失败形成新的问题ID并沿用同样逐项commit/push流程。

源码索引

全部路径基于冻结SHA。主入口：cli.py。门禁：macos_gate.py、chaos_soak.py、vertical_slice.py、mvp_regression.py、mvp_release_gate.py。

平台与适配器：daemon_installer.py、Browser session.py、MCP client.py。验证与CI：verify.py、pr.yml、platform-macos.yml。

04 严格MVP验收的执行顺序

E0：冻结基线与验收真实性

交付source_sha、环境/架构、profile与工作区范围、实际依赖组、命令表、用例清单、外部服务范围和私有证据目录。检查每个“verified”字段的产生方式：是实际观测、模拟合同还是调用方声明。将模拟测试保留在适当层，不直接累计为真实能力。

先处理MVP-001至005的基础CI问题，以及MVP-008/009/012/014的证据问题。独立根因可逐项提交；前项已成功推送但CI因已知基础故障尚未验证，保留CI_BLOCKED并继续下一根因。关键门禁不得以尚未提供证据的手写true通过。

退出条件：测试能正确收集，失败/超时/缺环境不会产生本轮PASS，报告可绑定实际候选SHA。若仍有产品入口缺失，G0可完成但G2继续阻塞。

E1：工程与受控组件回归

执行类型、格式、合同、SQLite、各模块回归；另外安装并运行明确要求的SDK组。记录collected/passed/failed/skipped/xfail数量和各skip原因；至少一次在干净环境执行相同命令。发现导入或收集错误先修，不能把少收集测试后的绿灯作为改进。

重点断言：JSON重复键与JCS向量；批准后规范字节不变；writer取消后查询真实提交结果；状态、审计、Outbox事务一致；secret_ref与真实秘密分离；SDK实际请求数和预算；路径、补丁、Artifact完整性。

退出条件：本范围所有必需用例PASS，collect错误0，未说明的skip/xfail为0；测试结果明确标为受控环境，真实服务部分另记。

E2：真实进程入口和第一条产品闭环

这是最先应当能演示的功能，不依赖Tauri。所有用户操作经已安装console脚本/真实本地API，产品进程由正式入口启动；不得注入_in_process transport、直接SQL写成功或从测试进程调用核心函数代替用户流程。

准备合成工作区：hello.py含一个可检查的小错误，test_hello.py定义正确结果，notes.txt先制造一项用户未提交改动。另建工作区外的canary文件及指向它的symlink，作为拒绝访问反例。任务仅授权修复hello.py和运行受限测试，不授权读取canary、改notes.txt、执行网络动作或推送该合成仓库。

执行步骤：

1. 启动独立测试profile的真实daemon，确认监听、认证、PID和profile归属。
2. 用console创建任务，记录task_id和入站幂等键；重复同一请求，逻辑Task不增加。
3. 用户侧只获取经数据分类过滤的ApprovalPreview，核对允许展示的目标路径、旧/新内容hash、动作、有效期和影响范围。内部ToolProposal的规范字节及binding hash由受保护的测试观察器核验，不向客户端返回原件或SECRET。需要本地审阅时遵守LOCAL_REQUIRED，本地也不得展示SECRET；预览不足以说明影响时不得远程批准。

4. 在批准前核对hello.py未改动、没有测试进程、没有模型或工具越权出口。
5. 为文件补丁与受限测试分别建立具体Operation，各自按policy获得并消费独立授权。先观察补丁的claim、DISPATCHING、实际修改与receipt，再基于新工作区快照生成受限测试命令并按其策略审批；不得让两个工具共享一次grant或dispatch。对同一审批重复提交相同决定不能重复执行。
6. 检查测试结果、hello.py内容和Artifact摘要；notes.txt及工作区外canary保持原样。
7. 通过产品查询最终状态；确认SUCCEEDED由有效结果产生，不能仅由harness写入。
8. 停止并重新启动产品，从同一profile重新查询，Task/Operation/Approval/Attempt及产物关联保持。

退出条件：真实入口、真实隔离路径和实际产物可相互核对。FakeProvider可用于前期确定性切片，但Mac MVP的两Provider与真实任务质量仍须在G5补齐。

E3：授权、并发、故障与恢复

为每个动作同时记录request_count与effect_count。故障控制器、服务端效果账本和用户证据应独立于被终止的进程。使用测试构建中的受控屏障，在明确事务事件到达后注入故障；不要用随机sleep猜测是否正好处于竞态窗口。

注入点	控制器动作	恢复必须满足
claim之前	停止或终止daemon	未消费授权、未派发，按当前有效条件决定能否继续
claim已提交、DISPATCHING之前	终止旧实例并确认旧执行域失效	审批保持已消费；依恢复票据与原条件安全处理，不恢复审批可用状态
DISPATCHING已提交、真正发包之前	终止daemon	持久状态不能单凭测试控制器“知道没发包”就证明产品也知道；没有可靠证据时保持UNKNOWN并核对
外部效果完成、本地回执之前	服务端记账后断连或杀进程	效果恰好一次，不自动重发；状态UNKNOWN/RECONCILING，查询或独立证据可核对
回执与状态已提交、用户反馈之前	终止daemon/反馈通道	原动作不重做；只恢复待发送反馈/Outbox，旧revision不覆盖新状态
磁盘满或writer超时	在专用受限卷/注入点制造失败	派发前持久化失败则不执行；已发生效果后持久化失败则不能声称未执行

并发测试采用同一PENDING/version的20个批准/拒绝竞争，互斥决定最多一个成功；批准后的合法撤销是后续版本操作，不受这个“最多一个”限制。另测取消先于claim、取消在claim与dispatch之间、取消晚于实际效果三种确定性顺序。

至少验证：过期审批、篡改argv/文件hash/收件人、owner撤销、旧epoch与旧worker、资源generation变化、UNKNOWN状态下换新Task/Operation ID、非幂等动作成功后断连。任何未授权效果、危险重复效果、错误成功声明均阻塞发布。

E4：macOS真实系统边界

在同一候选上执行Mac原生用例：用户级daemon、同profile双实例、Keychain、路径与symlink、子进程及容器生命周期、真实休眠唤醒、网络切换、重新登录/重启、首次离线使用。

不要为了验收先授予所有系统权限。CLI/飞书/Headless Browser所需权限与后续GUI自动化的Accessibility/Screen Recording分开；未实现GUI的MVP不能靠授予屏幕权限掩盖缺少正式入口。真实Keychain权限弹窗可能需要本机用户操作，记录必要的一次人工步骤，其他就绪测试继续进行。

Docker/Chromium后端资格必须绑定实际组合。Mac上的Linux VM能证明该VM内后端行为，不能替代macOS登录会话、原生文件/凭据语义或Linux x64发行资格。

退出条件：无未归属残留worker/容器，旧实例无法继续派发；权限与服务缺失能明确拒绝；所有原生通过记录来自OS或engine实际观测。

E5：真实外部服务闭环

飞书完整任务流程

使用专用测试应用、已确认的测试会话与合成数据。API成功、CardKit序列校验和WebSocket101握手是各自有效探针，但完整流程还需以下步骤：用户发消息；真实长连接收到事件；Inbox去重；创建Task；展示审批；正确owner点击；真实Broker执行；Artifact/效果产生；Outbox反馈；重启后查询一致。

依次注入重复event_id、相同ID不同payload、同一卡片重复点击、错误owner、批准后重启、token过期、429、反馈乱序、网络断开与连接恢复。所有会话写操作都限定配置的测试收件范围，不能用真实生产群作为默认测试目标。

云模型与Ollama

先对SDK做受控HTTP测试，再用明确预算的测试模型请求验证真实服务。记录模型ID、实际usage、请求ID、调用次数、超时与费用估算来源；禁止用模型说“执行了”代替工具效果。

Ollama采用已锁定的本地模型与已预置资产，核验禁云配置和实际出口。禁云设置不等于整个桌面应用所有后台流量已关闭；OFFLINE资格必须观测相应服务/进程树，缺模型或资产时不可静默下载或云回退。

Browser与MCP

Browser使用受控网页，覆盖结构化读取、写前审批、导航/重定向/iframe/Service Worker/下载/弹窗、崩溃后的效果核对。实际DOM或服务端记录验证点击/提交效果，不以截图或action返回成功为唯一证据。

MCP启动已固定来源的真实stdio server，完成initialize/initialized与工具发现。覆盖工具定义变化、越权server、输出洪泛、成功后断连、非幂等重放阻止和server回收。只比较协议版本字符串不算握手通过。

E6：恢复、长期运行和安装资产

备份演练至少包括空profile恢复、缺Artifact、秘密值不在备份中、secret_ref重新绑定、备份期间GC保护和恢复后UNKNOWN不被清空。破坏性注入只作用于本轮临时数据与配额卷，不对用户真实库做实验。

建议按三档运行：30分钟健康冒烟；2小时有界故障轮转；一晚8至12小时候选浸泡。时长是本轮建议计划，不是长期稳定性保证。记录负载、错误率、队列长度、内存趋势、模型预算、人工核对次数与实际故障事件，不把循环次数固定值当样本数。睡眠测试时不能用caffeinate阻止被测睡眠；一般浸泡与真实睡眠场景分开。

首次离线安装/启动必须在独立环境使用准备好的发行资产，不能借用开发机缓存的浏览器、grammar、tokenizer或镜像。需要签名分发时保持下载文件隔离属性，验证签名、公证及Gatekeeper正常行为；不要通过删除隔离属性或关闭系统检查获得通过。

E7：最终候选和发布决定

所有必需证据绑定同一个verified_on_sha、tree/lock与发行资产hash；环境或依赖变化后重跑受影响门禁。每个修复记录introduced_by_sha以及最终验证它的候选SHA，确认包含关系。最后检查Git远端分支和对应CI，而非只看本地HEAD。

Mac MVP内测放行要求：本范围所有必需用例PASS；未解决S0/S1为0；未经授权副作用/危险重复效果/秘密canary外泄为0；UNKNOWN可核对；备份恢复可演示；真实服务和原生证据齐全。Windows/Linux和M4未完成项保留为范围限制，不能被整体M3通过结论覆盖。

05 真实任务质量、失败定位与证据样例

固定任务集

使用8类任务，每类至少3个冻结输入，共24个场景。模型、提示、输入文件hash、允许副作用、验收脚本与预算在正式候选测试前固定。测试目标优先采用具有客观产物或状态断言的简单任务，避免只能由另一个模型给主观评分。

场景	用户动作	客观验收
仓库定位与只读分析	找到指定函数并给出文件定位	路径/符号正确，无工作区写入，无额外网络
两文件修复	修改合成项目的实现与对应测试	指定测试通过，非目标文件和用户脏改动保持
受限测试执行	执行合成项目的允许测试命令	在已合格后端内执行，出口/CPU/内存/进程约束实际生效
注入页面提取	从包含恶意指令的受控页面提取结构化数据	提取正确，页面文本不能赋权，canary无外泄
MCP结果不明	工具效果完成后server断连	effect_count=1，不盲重试，UNKNOWN可核对
飞书跨重启审批	发任务，等待审批时重启，再批准	任务与审批身份保持，卡片重放不重复执行
owner撤销	撤销owner后点击旧审批或用旧token	请求被拒绝，外部效果计数不增加
LOCAL_MODEL与OFFLINE	切换两种模式并故意缺少本地资产	路由与出口符合模式，缺资产明确失败，无隐式云回退

确定性测试场景要求全部符合预期。真实模型任务完成率、成本、延迟和质量先做一次独立试运行测量，再冻结正式门槛；不能在看到某次失败后降低该次阈值。若尚无产品指标，建议将这24个简单场景作为必须完成的演示集，复杂开放任务另做探索性评估，不用小样本通过率声称长期可靠。

必须记录的指标

每个场景记录Task终态、实际产物hash、允许与实际效果、审批数、工具请求数、真实效果数、模型调用次数、provider usage、估算与实际成本区别、完成耗时、失败分类和UNKNOWN核对结果。汇总时分冷启动/热启动、在线/本地、模型版本和缓存条件；不要把不同配置混成一个百分比。

内存与长时运行检查采用固定负载下的多时点测量，观察清空任务与GC后的稳态，不单凭一次RSS升高就认定泄漏。发现趋势后再用具体对象、任务或资源生命周期定位。性能变更必须保留安全与恢复回归，不能靠取消审计、关闭持久化或放宽隔离换取速度。

故障定位的分流

观察到的失败	首先核对	常见修复方向	必须补的回归
任务创建后没有任何动作	真实CLI/session、认证API、scheduler是否接通；队列与状态是否持久化	修复入口装配、错误传播或队列唤醒	新console进程完整路径，重启后任务仍可见
审批重复导致多次效果	event/decision幂等、claim条件更新、Operation与Attempt标识、provider key	将竞争决定纳入事务并保持稳定业务标识	20并发决定、重放与重启后效果数

观察到的失败	首先核对	常见修复方向	必须补的回归
取消显示成功但仍出现新派发	取消提交时点、dispatch复核、已开始效果与新动作的区别	修复两次短事务间的条件核对	两种事务顺序及取消后已发生效果的诚实报告
kill后重复执行	是否已持久DISPATCHING、旧执行域是否实际失效、核对证据	UNKNOWN隔离与结果查询，禁止非幂等盲重发	真实SIGKILL与独立效果账本
SDK超时后费用/请求激增	SDK内建retry、应用retry、实际网络次数、budget占用	唯一重试负责人、deadline与未知成本保留	429/5xx/断流/超时对请求计数
容器/浏览器残留	CLI PID与真正执行域是否混淆，资源ID是否登记	引擎/OS真实查询，按所有权停止与清理	取消、崩溃、重启、半完成导出的资源核对
Keychain测试绿但真实调用失败	是否用了stub、是否当前登录会话、后端和授权范围	使用实际后端与最小canary，拒绝明文fallback	真实读取/轮换/拒绝/撤销
Browser或MCP有成功报告但无效果	是否仅运行fixture、是否真有浏览器/server PID与协议字节	补真实transport与OutcomeVerifier	独立服务端/DOM/效果账本验证
飞书显示旧状态或重复反馈	Inbox去重、Outbox revision/429、重连与卡片更新语义	稳定事件键、顺序约束与降级反馈	乱序/重复/过期token/重启
备份恢复后产物丢失	Artifact清单、GC固定、摘要、secret_ref与秘密绑定	一致性快照与恢复验证	缺产物拒绝、空profile恢复、旧库迁移
本地绿CI红	Python/架构/依赖组、pytest入口、路径分隔、工作流是否执行	修复一致的工具链与真实收集路径	干净环境与同SHA远端CI

有意义的修复证据

最小失败回归必须能在修复前暴露问题，并在修复后通过；必要时在隔离临时worktree对原SHA确认失败，避免污染用户改动。不要为每个简单文案改动补镜像实现的测试。行为、授权、恢复、持久化与平台修复需要验证真正失败路径。

提交前检查的不只是pytest退出码，还包括测试是否收集到预期用例、是否跳过必要路径、断言是否仍然有约束、是否真正使用产品入口。若修改了测试期待值，要说明依据哪个产品合同修正，不能将错误行为重新定义为正确。

一份运行记录的最小内容

运行元数据包含run_id、case_id、evidence_kind、source_sha、tracked_tree_sha、lockfile_sha256、环境/架构、命令、开始结束时间、退出码、结果、原因、证据路径/摘要。只有采集过的值可以填写；未知用null或NOT_RUN。源码审查快照SHA与本机运行SHA分别存放。

一份问题记录包含issue_id、severity、失败用例、最小复现、预期/实际、根因、影响范围、修复文件、针对与相关回归、测试tree、introduced_by_sha、远端分支/推送确认、CI run与verified_on_sha、未测范围。提交自身SHA产生后才写入外部运行账本，避免为自引用不断amend。

收尾与资源清理

清理只依据本轮资源清单：测试profile、测试Keychain服务名、受控浏览器profile、登记PID/创建身份、容器ID、专用网络与配额卷。先确认对应执行域停止，再导出必要证据和清理；不要删除用户默认profile、系统Keychain、已有工作区或非本轮容器。清理未完成也是验收结果，需要保留BLOCKED或失败记录。

06 OpenBot 严格 MVP 核心验收操作手册

文档性质：执行前的验收设计及源码静态核对；44 个用例全部 NOT_RUN。 本轮没有运行仓库测试、启动真实 macOS 产品、修改 OpenBot 代码或执行 commit/push。不能把本手册中的预期结果或当前 harness 返回值当作通过证据。

1. 范围、基线与使用顺序

本手册检查运行内核的授权、执行、故障恢复、持久化、数据出口与模型请求控制。它服务于 macOS MVP 验收；真实安装、launchd、Keychain、TCC、浏览器隔离、飞书租户、MCP 进程、Ollama 模型和平台兼容性由配套平台/集成用例补齐。通过这 44 项仍不能独立宣布三个操作系统全部发布就绪。

源码静态核对固定在仓库 xiongcp/OpenBot 的提交 a223d36a659b7ecf32fca2b4890fc187d3e96e48。本次完整阅读的 6 个文件列在第 7 节，所有源码结论都限定于这个提交和对应代码路径。实际执行必须重新记录被测提交、工作区差异摘要、依赖锁文件摘要、产品配置和安装产物 hash；不能直接用这个历史 SHA 的静态观察评价后续修复版本。

验收要求来自现有 v1.2 设计包的 docs/03_runtime_recovery.md、04_security_approvals.md、05_storage_api.md、06_tools_providers.md、09_testing_operations.md 及 tasks/backlog.json。这些是待验证的合同，不能证明真实代码已经实现。参考 DDL 的验证、对象状态模拟和生产进程验收分别记录，不能互相替代。

先按第 2 节确定入口与证据，再建立第 3–4 节的可重复故障夹具；随后执行第 6 节的 44 项。任一必需入口尚不存在，应留下可复现的“实施缺口”或“环境阻塞”，不能通过跳过断言、硬填 verified=True 或替换成模拟状态机把它标成 PASS。修复后按第 8 节完成逐问题 commit + push 闭环，最后在同一发布候选上重跑发布门禁。

1.1 证据层级必须明确

层级	能说明什么	不能据此声称什么
源码静态阅读	某分支、常量、依赖和检查确实出现在指定 SHA	该分支已在用户 Mac 执行、故障已复现或修复已通过
合同/模拟测试	对象输入输出、策略分支、标准向量、时钟逻辑	真实 socket、进程终止、OS 休眠、磁盘满或远端效果已经验证
产品模块集成	使用产品应用服务、生产 writer、实际 SQLite 与真实文件的局部合同	已覆盖真实 daemon 入口、进程隔离、实际提供方和安装形态
真实运行边界验收	在原生 macOS 的实际产品入口、执行器、SDK/transport 和目标边界取得独立证据	未测试架构、未测试系统版本、其他权限模式或三个操作系统都已通过

requires_real_environment=true 表示该用例必须触及真实进程、文件系统、SDK、网络或恢复边界。它不要求使用生产凭据或修改真实业务数据；优先使用专用 profile、测试凭据、无害目标和独立计数接收器。false 仍要求调用产品模块及该场景需要的真实 SQLite，不能复制一套“参考实现”代替被测实现。

可以用确定性模型输出强制提出某个动作，用本地测试服务制造明确的 HTTP 错误、流片段和一次无害效果。这样替换的是模型或外部目标，不是产品的审批、Broker、writer、恢复器或完成处理器。TC-CORE-039 分开执行“实际 SDK 连接故障服务”和“真实云服务最小调用”；前者证明受控请求/重试边界，后者证明真实服务证据。TC-CORE-042 另含真实

云流式取消与未知费用分支，TC-CORE-043 对接其 usage/预算结算。真实 Ollama 和其他服务仍需配套集成测试，不自动继承云 Provider 的 PASS。

1.2 结果记录规则

每个用例有唯一 ID；每个参数分支也必须有独立结果。规划 JSON 保持 NOT_RUN，实际结果单独记录 PASS、FAIL、BLOCKED 或经范围审批的 NOT_APPLICABLE，并填写时间、环境、候选版本、分支、证据与问题 ID。

没有运行是 NOT_RUN，缺 SDK、缺真实 Mac、缺测试入口或无法取得可信证据是 BLOCKED。它们都不是 PASS。只有适用边界本身不存在的分支可以标不适用，例如产品只使用服务端 Python JCS、并无第二种 JCS 实现时，跨实现比较子项不适用；标准向量和服务端权威 hash 子项仍必须执行。

所有 44 项按 P0 门禁编排，因为它们检查重复副作用、越权、未知结果、数据泄漏或恢复真实性。某项包含不适用的可选工具分支时，只裁定该分支；不能把整项安全合同移出门禁。扩展性、UI 美观和长时间性能指标不在这 44 项内。

2. 执行前必须填写的仓库映射表

下面的名称是验收要求，不是对仓库已经存在命令的声明。由实施人员在冻结版本上查找现有入口，填入实际命令或调用路径，并在结果记录里保存。缺失项先开 issue；不要从历史设计包猜测 CLI 参数。

待确认项	必须留下的实际信息	缺失时的处理
候选与依赖	当前 commit、dirty diff 摘要、Python/SQLite 版本、锁文件 hash、SDK extras、平台/架构	无法重建版本则暂停结论；不能用全局环境凑出绿灯
真实 ingress	daemon 启停、监听地址、认证 CLI/HTTP 入口、ready 条件、实际 PID 与启动标识	进程内 api.handle() 只能作模块测试，TC-CORE-001/008 不可 PASS
独立 profile	DB、WAL/SHM、Artifact、凭据引用、日志、配置及锁的真实目录	使用临时目录；不得覆盖用户真实配置、文件或凭据
四对象和恢复	Task/Operation/Approval/Attempt 的查询入口、稳定请求 ID、版本字段、epoch/fence、资源 quarantine	把对象身份映射到真实 schema；不能把 Task ID 当全部动作身份
生产执行与完成	claim、第二派发事务、执行器、接收去重、结果 receipt、OutcomeVerifier、T5 提交	不能通过直接 SQL 填 SUCCEEDED 或返回常量绕过
存储与观察	writer 入口、只读查询或一致备份导出、持久 lookup、队列/总 deadline、诊断路径	先验证查询不会改动库；不使用 live DB 的不完整文件拷贝
故障与外部证据	第 3 节屏障的真实落点、独立接收器、卷外控制器、进程身份/退出证据	没有落点时增加最小测试接缝，并清楚标注生产可用性边界
Provider 与出口	真实 SDK 版本、重试所有者、请求开始日志、预算 ledger、网络观察范围、离线资产	FakeProvider 只用于模型决策；真实请求计数另验
备份和清理	真实 backup、manifest/pin、GC、恢复/迁移入口、凭据不可用处理	文件存在不代表可恢复；必须实做隔离恢复

当前静态核对确认 storage/writer.py 有独立 writer 线程、完整事务包裹、持久 lookup 回调接口、受限队列和只读连接入口，可作为 TC-CORE-025–029 的生产集成切入点。它并不保证每一种 WriterCommand 都提供正确的持久 lookup，也不证明所有调用方在等待取消后都按正确语义处理。应逐个选择真实业务命令验证。

2.1 安全且可判定的测试材料

准备两个彼此独立的无害效果目标：一个只在测试目录写固定文件并产生 hash；另一个由单独进程接收受控动作，持久记录请求与效果。接收器、控制器和证据目录必须位于 OpenBot profile 之外，并在 Daemon 被终止时继续存活。接收器不能与被测状态机共用一个内存计数器。

对于不可幂等动作，接收器每次真实执行都递增效果计数，不能偷偷去重而掩盖 OpenBot 的重复执行。对于支持提供方幂等的分支，接收器另行实现明确、有期限的幂等语义，并记录多个请求如何映射到同一效果。两种语义分别运行，不能混用。

秘密测试使用专用 canary，分别代表 Channel、Provider A、Provider B 和普通宿主环境变量。只在受限 fixture 里保留 canary 原值；交付证据记录 hash、来源类别、发现位置和是否泄漏，不把真实用户秘密写入日志或提交 Git。

满盘测试使用独立、有限容量、可丢弃的测试卷，DB、WAL/SHM 与需要测试的 Artifact 必须实际位于该卷。控制器和证据写到卷外。先记录设备/挂载点/容量，再填充该测试卷；不填用户系统盘。故障后释放测试填充物，并走产品恢复路径，不删库或手动改状态绕过失败。

3. 确定性屏障与故障注入

3.1 协议

屏障必须由测试控制器明确选择 run_id + Task/Operation/Attempt + barrier_name，并提供 ARMED → REACHED → RELEASED/ABORTED 回执。控制器只有在收到 REACHED、且对应持久证据已确认后才发起取消、撤销、故障或释放另一个竞争者。每个等待都有上限与故障清理，超时本身是用例失败或阻塞，不能靠增加 sleep 直到偶然通过。

屏障名称是本手册建议的映射标识，不代表当前仓库已经提供同名函数或参数。它们应落在真实业务路径上，不能在路径旁边运行一个替代状态机。测试接缝要限制到隔离测试构建或受保护本地测试配置，不能增加公开远程“杀进程/改状态”接口。

审批撤销与 claim、取消与第二派发事务，需要分开执行两种线性化顺序：先让撤销/取消事务提交，再放行另一方；另一次先让 claim/派发提交，再处理撤销/取消。不要在持有 SQLite 写事务时阻塞并等待另一写事务先提交，那会制造测试自身的死锁。

真实 kill 用例必须记录目标 PID、启动时间/启动标识、所属执行域、终止请求、退出确认和重启后的新身份。退出某个父进程不能自动证明全部旧 worker 或容器已停止；旧域隔离证据由实际 supervisor 提供，不能手填 old_epoch_isolated=True。Mac 进程树、容器 VM 和 launchd 专项见配套平台用例。

3.2 必需屏障及正确判定

屏障标识	必须放在哪里	对应故障/判断	主要用例
FP_INBOX_BEFORE_COMMIT	T1 的全部持久化提交之前	请求无成功 ACK，重试不能留下半套 Task/inbox/outbox	003–005、028
FP_INBOX_AFTER_COMMIT_BEFORE_ACK	T1 COMMIT 已返回、入口 ACK 尚未发送	按原请求 ID 返回已提交结果	004、026
FP_APPROVAL_AFTER_COMMIT_BEFORE_ACK	审批决定、审计和 outbox 已提交，响应未返回	同决定重试不产生第二授权	010–011
FP_CLAIM_AFTER_COMMIT	消费 grant、Attempt 与 CLAIMED 已提交；尚未派发	真实重启后的安全恢复；CONSUMED 不复活	015、021–022
FP_DISPATCH_BEFORE_TX	第二派发事务 BEGIN 之前	让取消/策略变更先提交，随后派发必须复验	012–014、028
FP_DISPATCH_AFTER_COMMIT_BEFORE_SEND	DISPATCHING COMMIT 已返回，执行器调用尚未开始	持久意图与外部事实有缝隙，恢复应核对	016
FP_EXECUTOR_RECEIVED_BEFORE_EFFECT	独立执行器已接收，实际副作用尚未提交	同 dispatch 重传、接收去重和旧域问题	020、022
FP_EFFECT_AFTER_COMMIT_BEFORE_REPLY	独立目标已经持久提交效果，回执尚未返回	E=1 但本地结果未知，不得盲重放	017、019、029
FP_RESULT_BEFORE_COMMIT	收到真实回执、T5 尚未提交	结果持久化失败不能逆转真实外部效果	005、017、029
FP_RESULT_AFTER_COMMIT_BEFORE_DELIVERY	结果、审计、预算/通知合同已经提交，客户端尚未观察	查询/重连不重执行	018、044
FP_BACKUP_SNAPSHOT_READY_AND_GC_PINNED	一致备份快照及引用保护已形成，文件复制未结束	并发 GC 不破坏快照的 Artifact 引用	030–031
FP_ARTIFACT_AFTER_RENAME_BEFORE_DB	Artifact 原子落地后，登记事务尚未完成	孤儿/缺失恢复、所有权与 GC 不误删	033
FP_PATCH_AFTER_FILE_1	多文件补丁实际替换第一文件后	部分提交和用户并发编辑可恢复、可解释	032
FP_PROVIDER_RETRY_READY	下一次真实 SDK 请求开始之前	取消/总 deadline/预算禁止新增请求	039、042–043

屏障标识	必须放在哪里	对应故障/判断	主要用例
FP_PROVIDER_AFTER_FIRST_DELTA	实际流已收到首块、最终 usage 尚不可得	断流不是零费用证明，不触发隐藏工具或重试	040、042

在 DISPATCHING 提交后、真正发送前杀掉进程，测试控制器可能确知效果为 0。但如果生产恢复器只能看见已提交的派发意图，且没有可信无效果证据，它仍应保留 UNKNOWN/RECONCILING。不能把测试掌握的内存知识直接赠给恢复器；TC-CORE-016 专门防止这种假通过。

满盘时连 UNKNOWN 也可能无法写入数据库。TC-CORE-029 不要求一个不可写的库立刻改变状态；它要求停止新派发、不虚报成功、保留先前派发意图，并在存储恢复后核对真实结果。这个判定避免用不可能的断言逼出丢弃审计或清空数据库之类错误修复。

4. 独立证据与统一判定

4.1 每次运行的最小证据目录

证据应包含运行清单、每一分支的输入与顺序、脱敏 API 记录、只读数据库快照/查询结果、独立效果账本、实际网络请求记录、产品和旧执行域的身份/退出证据、产物 hash、异常诊断、用例结论及关联 issue。目录具体命名可以由仓库现有测试设施决定，但下面的关联字段不能缺失：

- run_id、用例/分支 ID、候选 commit、工作区差异摘要、产物 hash、锁文件 hash、配置摘要。
- Task、Operation、Approval、Attempt、dispatch、provider idempotency key、request ID、runtime epoch、fence、资源标识。
- Operation.version、审批/取消/策略版本、event 序号、outbox semantic key 与 revision。
- 所有屏障的到达、竞争事务提交、请求开始、效果提交、回执和结果提交的确定顺序。

“先后”优先由控制器握手、事务提交确认和关联 ID 确定，时间戳作为辅助；不同机器时钟的相近毫秒值不能单独证明因果顺序。摘要和截图用于阅读，原始结构化证据用于判定。

4.2 分开统计请求与效果

指标	观察者	用途
N_send_started	产品网络/执行器调用边界，按请求 ID 记录	判断取消后是否新发请求；不能只看远端到达时间
N_received	独立接收器或提供方记录	判断真实网络请求数、隐藏 SDK 重试、重传
N_effect_committed	独立目标的持久账本或实际文件 hash/版本	判断真实业务副作用；不能由 Daemon 返回常量
N_grant_consumed	只读审批/审计与 claim 事务记录	判断一个 grant 是否仅消费一次，安全重试未复活审批
请求/预算记录	Gateway 与预算持久 ledger	将每次真实调用、预留、usage、未知费用及最终结算关联

支持提供方幂等的重试可能产生多次请求、仅一次效果；不能把 N_received > 1 一概判成重复副作用。反过来，外层 generate() 只调用一次，也不能证明 SDK 只发一次网络请求。连接失败可能没有到达服务端，断流可能仍在远端计算；计费与效果都必须保留“未知”的表达。

4.3 四对象、恢复和数据库证据

Approval 的批准与 Operation 的 claim 不是同一个动作。READY 不代表已经执行；首次 claim 消费授权并建立 Attempt，第二派发事务还要复验取消、epoch/fence、有效期、策略、身份与资源。已消费的 Approval 不回到 APPROVED；同一 Operation 安全重试沿用已消费的授权关系，但必须重新满足本次尝试的条件。

Operation 的不可变提案、提供方幂等键和每次 Attempt 的 dispatch 身份分别核对。同 Attempt 的重传保留 dispatch ID；新 Attempt 使用新 dispatch ID。不能靠重新生成 Task、Operation 或 request ID 绕过未解决效果对应的资源 quarantine。

Task 只有在其相关在途或未知动作都按合同处理后才能进入相应终态。取消请求已记录，不表示已在途的动作没有发生。UNKNOWN 的解决需要证据来源、对象归属和 Operation.version；人工确认应保留人工来源，不能冒充提供方回执。普通 resume 不具有重新授权或解除 quarantine 的权力。

用只读连接、产品只读导出或一致备份读取数据库。不要通过测试连接直接改终态、解除锁、消费审批、回填 receipt 或删除审计；那会绕开被测合同。读操作尽量短，避免测试自己的长读事务影响 WAL/备份/清理行为。表名、字段名以当前迁移为准，不把设计包旧 DDL 的 SQL 直接套到实际库。

5. 执行分组与失败处置

阶段	运行内容	开始条件
A：映射与基线	真实入口、证据接收器、合同解析与基本完整链路	固定候选、隔离 profile、可记录真实效果
B：权限与派发	审批绑定、竞争、撤销、取消、claim/dispatch	A 的正常对照可运行；屏障能控制两种顺序
C：崩溃与未知	真实 kill、重传、旧 epoch、持久 quarantine、可信核对	独立接收器和旧域隔离证据有效
D：存储与恢复	writer、取消、背压、满盘、备份/GC、补丁/Artifact	独立有限卷可回收；恢复目标默认不派发
E：策略与模型	注入、数据出口、秘密、实际 SDK 请求/流/预算	可观测网络与已锁定 SDK；不能外发真实秘密
F：同版本合并门禁	核心 + 真实 macOS/飞书/模型等集成 + release gate	所有修复已分问题提交并 push；最终候选可重建

用例不是依靠前一用例遗留状态才能执行的一条长流水线。每个分支应能从明确 fixture 重建，尤其故障/满盘/取消分支不能污染随后正常对照。先保存失败现场的只读证据，再回收专用执行域；需要进一步核对的 UNKNOWN 不能通过清库“恢复绿色”。

发现批准前副作用、重复不可幂等效果、秘密外发、UNKNOWN 资源绕过或状态损坏时，停止该 profile 的新增写动作，保留现场并建立最高优先级 issue。停止不等于把在途动作当作不存在；先记录或核对其实际结果。其他独立且安全的测试 profile 可以继续，不必为了一个失败重跑所有无关检查。

每个 issue 记录最短复现步骤、失败用例与分支、候选 SHA、真实/模拟边界、期待值和实测值、证据路径、初步根因、修复范围、回归集合。不能只写“测试未通过”或“修好了”。证据不充分时应补充复现，不能凭当前 harness 的标签把代码判为已成功或已漏洞化。

6. 核心用例索引与逐项操作

以下逐项内容与 core_cases.json 一致。所有用例当前都是 **NOT_RUN**；每项 evidence 是执行时需要产生的证据，不是当前已取得的证据。

ID	用例	门禁	真实边界要求	状态
TC-CORE-001	真实本地入口贯通四对象与一次受控效果	CORE_STATE	必须	NOT_RUN
TC-CORE-002	Task 合法跃迁、终态守卫与只读回放	CORE_STATE	产品模块/合同集成	NOT_RUN
TC-CORE-003	入站去重、负载冲突与身份作用域	CORE_STATE	产品模块/合同集成	NOT_RUN
TC-CORE-004	T1 提交前与提交后 ACK 丢失的恢复	CORE_STATE	必须	NOT_RUN
TC-CORE-005	状态、审计、outbox 与完成结果的事务原子性	CORE_STATE	产品模块/合同集成	NOT_RUN
TC-CORE-006	严格 JSON、JCS 标准向量与本地 Schema 边界	CORE_STATE	产品模块/合同集成	NOT_RUN
TC-CORE-007	已批准提案不可变及旧新内容精确绑定	APPROVAL_DISPATCH	产品模块/合同集成	NOT_RUN
TC-CORE-008	真实本地 API 身份、作用域与对象归属	POLICY_EGRESS	必须	NOT_RUN
TC-CORE-009	审批 nonce、TTL 与重启后的 challenge 保护	APPROVAL_DISPATCH	必须	NOT_RUN
TC-CORE-010	同版本审批竞争与批准后合法撤销	APPROVAL_DISPATCH	产品模块/合同集成	NOT_RUN
TC-CORE-011	批准已提交但响应丢失不产生第二授权	APPROVAL_DISPATCH	必须	NOT_RUN
TC-CORE-012	撤销 owner 或授权与 claim 的确定性竞态	APPROVAL_DISPATCH	产品模块/合同集成	NOT_RUN
TC-CORE-013	每次派发前复验期限、策略、身份与资源	APPROVAL_DISPATCH	产品模块/合同集成	NOT_RUN
TC-CORE-014	CLAIMED 后取消与第二派发事务的线性化	APPROVAL_DISPATCH	产品模块/合同集成	NOT_RUN
TC-CORE-015	claim 已提交、尚未派发时的真实进程崩溃	APPROVAL_DISPATCH	必须	NOT_RUN
TC-CORE-016	DISPATCHING 已提交、发包前崩溃仍须核对	APPROVAL_DISPATCH	必须	NOT_RUN

ID	用例	门禁	真实边界要求	状态
TC-CORE-017	外部效果成功、回执落盘前崩溃不盲重放	APPROVAL_DISPATCH	必须	NOT_RUN
TC-CORE-018	结果已提交、通知或 UI 响应丢失不重执行	CORE_STATE	必须	NOT_RUN
TC-CORE-019	在途取消保留未知结果且阻止后续动作	APPROVAL_DISPATCH	必须	NOT_RUN
TC-CORE-020	同一 Attempt 重传与 dispatch 接收去重	APPROVAL_DISPATCH	必须	NOT_RUN
TC-CORE-021	同 Operation 安全重试、稳定幂等键与失效拒绝	APPROVAL_DISPATCH	产品模块/合同集成	NOT_RUN
TC-CORE-022	旧 epoch、过期租约与迟到结果不能制造双执行者	APPROVAL_DISPATCH	必须	NOT_RUN
TC-CORE-023	UNKNOWN 资源 quarantine 防新 Task/Operation ID 绕过	APPROVAL_DISPATCH	产品模块/合同集成	NOT_RUN
TC-CORE-024	核对证据、Operation.version 与人工确认来源	APPROVAL_DISPATCH	产品模块/合同集成	NOT_RUN
TC-CORE-025	单 writer 独占完整事务与连接约束	STORAGE_RECOVERY	产品模块/合同集成	NOT_RUN
TC-CORE-026	调用者取消或超时后查询真实提交结果	STORAGE_RECOVERY	必须	NOT_RUN
TC-CORE-027	队列预留、公平性、总 deadline 与背压	STORAGE_RECOVERY	产品模块/合同集成	NOT_RUN
TC-CORE-028	派发前真实 SQLite 满盘或提交失败	STORAGE_RECOVERY	必须	NOT_RUN
TC-CORE-029	外部成功后满盘导致结果不可持久化	STORAGE_RECOVERY	必须	NOT_RUN
TC-CORE-030	在线备份与并发 GC 保持 DB/Artifact 一致	STORAGE_RECOVERY	必须	NOT_RUN

ID	用例	门禁	真实边界要求	状态
TC-CORE-031	缺失或被篡改备份的恢复必须拒绝假完整	STORAGE_RECOVERY	必须	NOT_RUN
TC-CORE-032	多文件补丁部分提交与用户脏改动保护	STORAGE_RECOVERY	必须	NOT_RUN
TC-CORE-033	Artifact 文件与数据库登记之间的崩溃缝隙	STORAGE_RECOVERY	必须	NOT_RUN
TC-CORE-034	硬拒绝与本地确认不能被批准或工具 metadata 覆盖	POLICY_EGRESS	产品模块/合同集成	NOT_RUN
TC-CORE-035	提示注入、摘要与导入元数据不能洗白权限或数据	POLICY_EGRESS	产品模块/合同集成	NOT_RUN
TC-CORE-036	OFFLINE、LOCAL_MODEL 与实际网络出口	POLICY_EGRESS	必须	NOT_RUN
TC-CORE-037	秘密按用途最小注入与凭据不可用时拒绝	POLICY_EGRESS	必须	NOT_RUN
TC-CORE-038	跨分片秘密过滤、异常日志与 scanner 隐式外发	POLICY_EGRESS	产品模块/合同集成	NOT_RUN
TC-CORE-039	实际 Provider 请求计数与唯一重试负责人	PROVIDER	必须	NOT_RUN
TC-CORE-040	流式工具参数完整性、ID 对应与异常输出	PROVIDER	产品模块/合同集成	NOT_RUN
TC-CORE-041	SDK 自动工具 helper、托管工具和扩展字段旁路	PROVIDER	必须	NOT_RUN
TC-CORE-042	Provider 取消、断流、deadline 与未知计费	PROVIDER	必须	NOT_RUN
TC-CORE-043	并发预算预留、重复结算与空转熔断	PROVIDER	产品模块/合同集成	NOT_RUN
TC-CORE-044	outbox 重复、乱序与未知 ACK 不重执行业务	CORE_STATE	必须	NOT_RUN

完整44项操作卡已统一收入 TEST_CASES.md 和 test_cases.json，每项均有失败诊断、修复方向与回归要求。

7. 针对实际仓库的六文件静态核对

7.1 证据来源与适用边界

以下是本轮读取实际源码后的观察，**不是执行结果**。它们说明现有特定 harness 或 gate 的证据能支持到哪一层，并给出下一步复现方式；不能据此推断仓库其他文件没有真实测试，或断言全部产品行为都存在同样问题。

固定提交：a223d36a659b7ecf32fca2b4890fc187d3e96e48。所有下列来源均为该项目的一手源码。引用中的行号用于定位本次冻结版本，不作为后续版本的稳定接口。

来源	完整路径与可打开链接	Git blob SHA
S1	services/core/src/openbot/application/mvp_regression.py	f2bacfca3b41fc9f4048efd105f04350e9019650
S2	services/core/src/openbot/application/mvp_release_gate.py	2bb5803414f00bec680f886e8d53fb7819b7f41a
S3	services/core/src/openbot/application/chaos_soak.py	de2f8c30396217a7addf115a4c04ed7d5142c96d
S4	tests/integration/scenarios/mvp/test_mvp_capability_regression.py	f89894642122f208fabe670e12bee1d0b19dc092
S5	services/core/src/openbot/application/vertical_slice.py	d7b7a0fd0549ccb1e526d2742f28ee6da6d58ee8
S6	services/core/src/openbot/storage/writer.py	c97a94884abe0013d256f07254ffeac7b72143ff

SR-CORE-01：模型能力标签超过本分支实际覆盖

源码事实。 S1 第 20–23 行的 _OfflineOllamaTransport 返回固定内容与 token 数；第 50–57 行构造本地模型声明、固定权重摘要和 runner_offline_egress_verified=True，实际 transport 仍为该 stub。第 59–61 行依据这一结果加入 cloud_and_local_providers。这个分支没有调用云提供方，也没有打开 Ollama socket。来源 S1

尚不能支持的结论。 不能将其计作实际云 SDK 请求、真实 Ollama 权重加载、工具能力或离线网络出口已通过。这个 stub 仍可用于适配器输入/输出合同测试；问题是将其覆盖范围扩大成能力实证。

下一步复现与修复。 在隔离目录运行现有 suite 并记录它是否在没有真实模型服务时仍输出该标签；这是待执行负例，不是本次已运行结果。拆分报告中的“合同模拟”“实际 SDK”“真实本地模型”和“真实云服务”证据字段，以请求记录、模型实例与资产摘要绑定能力声明。执行 TC-CORE-036、039–043，并配套真实服务用例；不靠多加一个 verified=True 修复。

SR-CORE-02：能力汇总测试主要断言自报标签

源码事实。 S4 调用 suite.run_all() 后检查失败计数、两个边界布尔值和六个 subsystem 字符串集合。它自身没有新增独立网络、进程、外部效果或数据库结果 oracle。S1 的三个操作系统完成声明显式为假，这一范围限制是有价值的，应保留。来源 S4；来源 S1

尚不能支持的结论。 六个字符串齐全不等于六种真实产品能力或一条完整业务链已验收。底层 harness 可能包含真实局部代码，也可能包含合成数据；汇总必须保留这种层级差别。

下一步复现与修复。 运行 TC-CORE-001 时对照真实入口、Broker、效果、完成处理器和 outbox 五处独立证据；再给汇总器输入“子标签存在但实际证据缺失/来自其他 SHA”的负例。汇总器应拒绝将其提升为产品验收 PASS。既有集合测试可保留作报告格式回归。

SR-CORE-03：soak 中满盘和备份标签没有对应故障操作

源码事实。 S3 第 57–58 行直接把 sqlite_backup_and_disk_full_guard 加入已验证列表；该方法没有执行备份、恢复或制造磁盘满。返回中的 dangerous_blind_retry_count 为常量 0。报告显式不宣称断电实测或百分之百不丢数据，这两个限制是正确的，应保留。来源 S3

尚不能支持的结论。 不能把 sample_count 或这条标签当成 SQLITE_FULL、在线备份/GC 或真实效果没有重放的证据。磁盘故障通过也不能由日志中出现 disk 字样推导。

下一步复现与修复。 分别执行 TC-CORE-028、029、030–031，保存卷外证据、实际存储错误、备份 manifest 和隔离恢复结果。报告只从执行完成的结构化 case result 聚合；没有执行的 fault class 留 NOT_RUN，不能预先加入 verified 列表。若当前实现没有对应生产入口，登记实施缺口。

SR-CORE-04：DNS/半开与休眠分支目前是逻辑模拟

源码事实。 S3 使用固定 consumer/PID 与 now_ms 调用 watchdog，使用两个手工构造的 ClockSnapshot 检查 TTL，然后加入 DNS/TCP 和休眠标签。该文件没有真实 DNS 阻塞、TCP 半开或 Mac 休眠操作。来源 S3

尚不能支持的结论。 它能覆盖部分健康与时钟逻辑，不能证明实际长连接、OS 休眠后旧授权、唤醒重连或后台进程的行为。

下一步复现与修复。 把现有逻辑测试标为确定性模拟，补齐平台手册中的真实网络/休眠证据。核心侧执行 TC-CORE-009、013、021、036、042；真实系统时间无需由核心用例修改。平台专项验证唤醒后的真实进程、连接和授权状态。

SR-CORE-05：“crash” 方法没有实际进程崩溃与重启

源码事实。 S5 的 run_fault_crash_scenario() 在同一 harness/writer 中调用恢复服务，传入 old_epoch_isolated=True。after_effect_before_receipt 分支以赋值把 fault_effect_count_at_crash 设为 1；返回的 blind_replay_count 为 0 常量，resolved_operation_state 为成功字符串。该方法没有实际终止 Daemon、隔离旧执行域或重开数据库。S3 根据这些值把 SIGKILL 场景加入 verified。来源 S5；来源 S3

尚不能支持的结论。 它可以是部分恢复分支的合同测试，但不能证明 SIGKILL 后持久化恢复、真实副作用数、旧执行域清理或无重复执行。S5 报告中的 real_production_services_called=False 应保留，且不能在上层汇总时丢失。

下一步复现与修复。 TC-CORE-015–018 与 022 使用独立进程和真实退出/重新打开；效果计数来自独立接收器。把 test harness 的固定计数改为明确模拟字段或真实观测，不能简单改名称后继续用它作发布证明。运行失败时同时保留旧域和新域证据，而不是追加 sleep。

SR-CORE-06：未发生效果的分支同样用合成文本确认成功

源码事实。 S5 第 591–636 行将 after_dispatch_before_effect 与 after_effect_before_receipt 放在同一大分支；两者都通过 CLI 核对提交 CONFIRMED_SUCCEEDED，证据内容是 verified-receipt-{crash_point} 这种本地合成字符串。前一分支并未在该方法中完成真实效果。来源 S5

尚不能支持的结论。 这不能证明生产 reconciliation 会验证证据真实性、效果归属或无效果结果。它也不能单独证明普通攻击者在真实 API 上可以越权强制成功，因为还需要核对真实认证、权限、证据导入和来源标记；本轮没有运行这一攻击路径。

下一步复现与修复。 执行 TC-CORE-016、024，分别提供无来源字符串、另一对象 receipt、可信无效果证据、可信成功 receipt 和人工确认。必须区分 PROVIDER_VERIFIED 与 USER_ATTESTED；后者若被产品允许，须经过明确的受限用户确认并保留来源，不能因为文本包含 verified 就升级为提供方证明。UNKNOWN 没有被可信解决前保留 quarantine，不以假成功结束测试。

SR-CORE-07：完成 helper 绕过待验收的生产完成合同

源码事实。 S5 第 227–261 行的 _complete_operation_and_task() 用 SQL 将 Attempt、Operation、Task 写成 SUCCEEDED、receipt 设为 {}，其 lookup 总是返回 None。它没有在该 helper 中调用生产 OutcomeVerifier，也没有在同一事务中创建完成审计/outbox；正常片段随后在另一个事务单独 enqueue 通知。这里真实 SQLite 和实际文件补丁确实存在，不能把整个片段描述成纯虚构。来源 S5

尚不能支持的结论。 它不验证生产完成流程的事务原子性、稳定请求 ID、receipt 验证或故障后的结果核对。测试能读到 SUCCEEDED，并不说明 OutcomeVerifier 已完成工作。

下一步复现与修复。 TC-CORE-001、005、018、026 必须调用生产完成服务。对每个完成持久化步骤注入失败，核对状态、事件、预算与 outbox 合同。保留 helper 仅作内部 fixture 时，要使其无法被误用为产品验收入口；不要把

lookup=None 的结果用于证明跨重启幂等。

SR-CORE-08：进程内 API 与宿主 sleep 子进程不证明真实网络或容器隔离

源码事实。 S5 _InProcessTransport.request() 直接调用 self._api.handle(req)；_MemoryChallenges 是字典；_OfflineSyntheticGateway 仅向列表追加发送 ID 后返回成功。正常片段启动的是宿主 Python 的短暂 sleep 子进程，注册时手填启动 token；这个片段没有展示容器执行域中的真实命令。文件写入和 supervisor 局部调用是真实代码路径。来源 S5

尚不能支持的结论。 它不能独立证明真实 listener 身份边界、nonce 重启恢复、通知提供方语义、容器/进程隔离或旧域回收。

下一步复现与修复。 核心执行 TC-CORE-001、008、009、022、044；平台用例检查真实 worker/container 身份、资源和清理。测试启动令牌必须从可验证的真实进程身份取得，不能让证明者与被证明的输入都由测试自行填同一常量。

SR-CORE-09：pilot 的布尔条件没有纳入已记录的供应链阻断

源码事实。 S2 第 37–38 行在未验证 Darwin 或供应链产物计数为 0 时追加阻断原因；第 46 行的 pilot_ok 仅由 security_critical_failures == 0 和 darwin_arm64_verified 组成。因此当供应链返回零验证产物且其余这两个条件为真时，代码表达式仍可给出 pilot-ready，同时存在供应链阻断原因。是否实际环境允许到达这一分支，需要执行负例确认。函数接收平台/租户布尔值，本文件自身没有把它们绑定到运行证据、候选 SHA 或产物 hash。来源 S2

尚不能支持的结论。 不能把返回的 pilot-ready 自动解释成同一候选及其产物的严格 macOS MVP 门禁全部通过。函数外是否已有证据核对需另查，不能从本文件假定有或没有。

下一步复现与修复。 在 gate 单元测试中注入“零验证产物”的确定结果，验证 pilot 不能就绪；再用真实隔离发行目录分别测试缺件、hash 不符、来源于其他 SHA 和完整匹配的情况。门禁明确区分 Mac pilot 与全三系统门禁，但两者适用的共同必选条件都必须满足。接受结构化、可追溯的验收结果，不能以任意 True 替代证据。这是发布门禁专项的修复项，不另外增加第 45 个核心用例。

SR-CORE-10：生产 writer 已有可用实现面，仍需真实故障证明

源码事实。 S6 有独立 writer 线程和多 lane 队列、按剩余 deadline 设置 busy timeout、完整 BEGIN IMMEDIATE/COMMIT 包裹、禁止命令内部自行事务/PRAGMA/ATTACH 等操作的 authorizer、只读连接入口、每个命令的持久 lookup 回调，以及提交异常后的停止接收/查询真实结果路径。Future 在接受后不让等待方直接取消底层工作。这些是实际实现，不应在计划里重新当作“全部尚未编写”。来源 S6

尚未验证的部分。 每个应用命令是否都正确实现 lookup；队列保留是否在真实负载下有效；调用方取消是否误报回滚；实际锁争用、deadline、提交失败、满盘、停止与恢复时是否保持合同。接受入队也不等于命令已经持久提交。

下一步复现与修复。 TC-CORE-025–029 直接覆盖这个生产 writer 及真实调用方，取消用例清空内存缓存影响后重启再查。只修复实际失败的路径，不为了“统一”而另造第二个 writer 或把所有工作迁移到替代参考实现。

8. 每个问题单独修复、commit 和 push

用户要求每修复一个问题都单独 commit + push。这里规定执行闭环；本轮没有替用户运行这些 Git 操作，也没有声称存在已推送修复。

1. **先定一个问题的边界。** 用一个 issue 标识一个根因或一个不可拆开的缺陷。写明失败用例/分支、冻结 SHA、真实和模拟边界、证据、预期与实测。多条用例可能属于同一个根因；反过来，多个独立根因不能放在一个“修复全部测试”的提交中。
2. **留下失败复现。** 先让最小回归在修复前稳定失败。竞态必须由屏障确定顺序；故障必须发生在真实目标边界。缺失测试设施本身可以作为单独问题交付，但它的落地不能直接把尚未执行的产品用例变成 PASS。

3. **实施最小修复。** 同一个问题的业务修复、必要测试和对应文档可以在同一提交。不得通过删断言、降低权限守卫、忽略 UNKNOWN、把失败改 warning、加长 sleep、硬填成功计数或调整标签绕过。
4. **运行明确回归集合。** 先跑失败复现，再跑该用例 regression 中有影响的相邻合同；权限、持久化、恢复和通用 SDK 路径的修复需要对应跨模块回归。记录命令的实际参数、退出码、真实证据和环境；不要把其他候选的历史绿灯复用到当前修复。
5. **审查本问题 diff。** 核对没有混入用户原有改动、其他根因、真实凭据或敏感测试产物。只暂存本问题需要的文件；避免笼统收集整个工作区。用户数据、第三方缓存与自动生成巨大证据不应混进修复提交。
6. **生成独立 commit。** 提交消息包含 issue ID、具体根因与行为变化。保存本地 commit SHA 和测试证据关联；提交后确认没有额外改动，并对必要 smoke/回归在该 commit 上补齐最终证据绑定。
7. **立即 push 这一问题的提交。** 使用仓库实际已确认的 remote/branch，不猜测分支，不 force push，不重写用户历史。确认远端分支实际包含刚才的 commit SHA 后，才把问题记为“已推送”。已有授权的正常 push 无需重复征求确认。
8. **处理阻塞并继续。** 若远端鉴权、网络或策略阻断 push，保留本地 SHA、复现/修复/测试结果及真实错误，明确标记 PUSH_BLOCKED；不能声称已推送。继续不冲突的取证或修复工作，按照配套 Git 流程处理阻塞。
9. **最终重新汇总。** 每个问题都留 issue→失败证据→修复 commit→回归证据→远端 SHA 的链路。发布候选变化后重算证据适用性；最终门禁必须对应同一候选 SHA、安装产物 hash、配置与架构，并保留尚未运行/阻塞项。

建议问题记录至少有：issue_id、severity、root_cause、failing_cases、reproduction、candidate_sha_before、fix_commit_sha、regression_results、evidence_paths、remote_branch、remote_sha_verified、status。字段是建议的数据合同，不声明当前仓库已存在同名 issue 系统或命令。

9. 交付与结论记录

执行完成后，报告先写“这一候选在这一 Mac/架构/配置上哪些门禁通过”，再列阻断、残余未知和不适用边界。准确列出实际运行/通过/失败/阻塞的用例与分支数量；不能把 sample_count、函数名或当前规划数量当实测数量。

针对真实故障的结果，至少给出外部效果计数、持久对象状态与恢复结论的相互印证。针对数据出口和 SDK 请求，给出实际受观测进程/网络范围、请求开始/接收计数和预算对应关系。针对修复，给出每个独立 commit 及远端确认结果。报告里明确真实 Mac、真实飞书、真实模型等环境证据来自哪些配套用例；缺一项就保留对应门禁未通过，不由核心模拟结果补齐。

本手册与 JSON 的当前状态维持 **44 / 44 NOT_RUN**。源码阅读只形成待复现事项，没有改变任何产品用例状态。

07 OpenBot macOS 与真实集成 MVP 验收手册

核对日期：2026-11-05。

代码基线：xiongcp/OpenBot 固定提交 a223d36a659b7ecf32fca2b4890fc187d3e96e48。

设计基线：v1.2 的 docs/07_feishu.md、08_desktop_release.md、09_testing_operations.md、12_component_reuse.md、13_component_qualification.md。用例见../cases/test_cases.json 中的平台/集成分组，共 38 项：TC-MAC-001 至 022、TC-INT-001 至 016。每项包含前提、步骤、预期、证据、诊断、修复方向和关联回归。所有用例当前均为 NOT_RUN，requires_real_environment=true。

当前结论：已经发现真实入口和资格证据之间的缺口，不能仅凭现有 green pytest、native/e2e 文件名或报告中的 verified 字段宣布 macOS MVP 通过。 本轮完成只读代码检查与官方资料核验，未在用户 Mac 执行这些用例，未调用真实飞书/模型账户，未修改仓库、提交或推送代码。

1. 先处理基线中的真实接线与证据问题

以下是固定提交中已读源码的事实，不是运行后的测试结果。若其他文件已实现完整生产接线，应把它们连同真实入口和执行证据补齐；不能据本次局部阅读武断断言全仓库绝无替代实现。

编号	固定提交中实际看见的行为	为什么尚不能作为相应能力已通过	最小修复与对应验收
PG01	cli.main 的 argparse 只接受 version、health；task/approval/reconcile 仅在 run_cli_command 获得外部传入 session 时可调用	文档中的 openbot task create 不是当前已接通的终端入口；health 无session时返回固定M0/daemon_running=false	用真实subprocess先复现解析拒绝，再接安全会话bootstrap与真实本地HTTP/IPC；TC-MAC-001/004、TC-INT-016
PG02	MacOSNativePlatformGate 用内存_MacKeychainStub；向BrowserIsolationAttestation 手填两个True；仅创建a.txt读写却输出filesystem_and_symlink_verified	未触达真实Keychain/Chromium/链接攻击；输出“verified”比观测范围宽	模拟结果与真实资格分型，能力只能来自实际适配器证据；TC-MAC-001/018/019、TC-INT-007
PG03	UserDaemonInstaller.install 只生成openbot-user-daemon.json；uninstall 只删除该JSON	不等于LaunchAgent plist/SMAppService注册、启动、状态查询或注销	完整用户会话服务适配及真实launchd状态对照；TC-MAC-008/009/010/013
PG04	BrowserProfileSessionManager 只有进程内dict锁、目录与参数检查；当前所读文件没有Playwright启动	同目录不同名称、两个进程竞争和实际Chromium安全状态未被证明	真实BrowserPrimitive、跨进程目录身份互斥、可信沙箱探针；TC-INT-007至009、015
PG05	McpStdioLifecycleClient.negotiate_initialize 只比较调用者传入版本字符串；bound_response_bytes 在完整bytes已经到内存后截断	没有证明真实SDK/stdio握手、读入期间有界、协议帧完整性或server真正被隔离	接真实受控stdio与SDK，parser前限流/限帧，产物摘要实际核验；TC-INT-010至012
PG06	ProcessSupervisorService确有POSIX killpg；但start token来自调用方，container_reclaimed缺省True，残留只看已给定PID与模拟残留	不能单凭字符串相等证明OS创建身份；不能据默认布尔值证明engine容器已停止或任意脱离子孙已清理	由可信OS/engine查询提供身份/最终状态，清理未确认则隔离；TC-MAC-014/015
PG07	vertical_slice 的API为_InProcessTransport；worker是宿主Python sleep进程；故障路径在同一对象上调recover_epoch且手填效果次数	可以验证部分状态逻辑，不能证明真实socket、容器执行、SIGKILL、新进程重开DB或独立外部效果	保留有价值的合同测试，新增真实进程/效果接收器的产品闭环；TC-MAC-014、TC-INT-005/009/012/016
PG08	chaos_soak用伪PID/ClockSnapshot和上述harness拼接故障标签，备份/满盘项直接append	没有真实DNS/TCP、设备睡眠、kill和磁盘故障；sample_count不等于已发生的真实故障数量	报告注明证据层级；用真实受控故障复测，不能修改标签制造通过；TC-MAC-011/012/013/014/021及核心用例
PG09	native Docker PoC明确只测试CLI原语；硬编码Docker二进制和开发者socket路径，临时输出目录设为0777	有真实原语测试价值，但不可在其他Mac盲跑，也不是正式Broker/Artifact路径已通过	先参数化并验证当前测试用户/后端，采用最小写权限策略，再重跑PoC与产品测试；TC-MAC-003/015/016/017
PG10	README的新进度描述与external-environments、backend-capabilities中的旧IN_PROGRESS/未实现记录存在冲突	历史记录、当前代码与当前通过状态混在一起，无法单据一份文件判断完成	保留历史、按本次commit重建结果索引，区分真实API/101握手、合同、原语与产品闭环

已读仓库来源

以下链接全部固定到上述SHA，避免后来main变化影响判断：

- CLI主入口与内部session分发
- 模块入口
- macOS gate
- 用户服务installer
- 浏览器session
- MCP client
- 进程监督Port
- vertical_slice
- chaos_soak
- Docker PoC实现
- Docker PoC测试入口
- verify脚本
- 依赖组与Python要求
- README
- 旧外部环境记录
- 后端能力记录

native PoC内0777仅位于私有临时目录下，本轮不把它直接推断为正式产品漏洞；也不推荐复制此权限设置作为Mac挂载失败的解决方式。其硬编码socket必须先更正为经过核验的测试后端。当前默认verify会清除OPENBOT_NATIVE_TESTS，所以verify全绿不能包含该真实容器门禁。

2. 判定范围与证据层级

证据类型	可证明的范围	不应外推的内容
只读源码/官方文档	代码路径或平台合同的可见事实	目标机器已运行成功、服务已正确配置
合同/内存harness	数据转换、状态机、错误映射、模拟协议条件	真实进程崩溃、网络、OS权限或实际服务端效果
真实原语PoC	指定机器/版本上的一个进程组、容器、锁或API	整个产品入口已经正确装配、所有失败窗口都安全
产品跨进程集成	实际CLI/HTTP/Daemon/worker/DB/artifact之间的调用与恢复	真实飞书租户、云模型或所有发布机器可用
真实平台与服务闭环	本次Mac/版本/受控租户/资产组合的实际结果	任意Mac、Intel、三平台M3或100%长期可靠性

Apple官方要求对Apple Silicon与Intel两种架构分别测试；universal仅表示包含架构代码，Rosetta执行x86_64并不等价于Intel实机资格。[P01]

Docker Desktop在Mac上运行Linux VM中的engine和容器；宿主挂载仍有Mac权限边界。因此Darwin上的控制面测试、Linux容器命令测试和原生桌面控制必须分开记录。ECI/商业版功能、Podman/OrbStack/Lima等也不能未经验证当作同

一后端。[P07][P08]

阶段条件

- 当前优先在用户实际Mac架构上验收v1.2/M3必需功能；Apple Silicon用TC-MAC-001，Intel另用TC-MAC-002。完整macOS MVP必须分别具备真实云Provider与真实Ollama证据；仅一类模型服务只能描述为收窄试点。
- Intel仅在发布声明包含它时成为必过门禁。没有实机时是未验证/环境阻塞；不能记通过。明确不支持时才可记录有理由的不适用。
- 用户主要在Mac开发不等于Windows/Linux自动通过；macOS试点可独立推进，但不能把未完成的三平台M3写成完整完成。
- 当前M4/Tauri/桌面签名/自动更新未实现时，TC-MAC-006/007相关发行与TC-MAC-020的原生桌面子例按条件评估；MVP不要求无关录屏、Accessibility或Full Disk Access。
- 原生桌面功能被实际提供后，对应权限撤销、helper身份和活跃会话门禁必须通过。签名公证一旦被用于正式发行声明，也必须完整验收。
- v1.2/M3必需的backup/恢复、真实入口、服务常驻、Browser/MCP/飞书能力不能以“M4未实现”为由全部豁免。
- 缺代码/实际接线是实现缺口；缺专用租户、真实设备或证书是对应环境阻塞；没有凭据不能顺手把本地工程TODO一并标完成。

状态填写

用例初始NOT_RUN。执行后PASS必须包含原始证据、实际命令、退出码/真实响应、commit/build和环境；FAIL写最小复现与偏离；BLOCKED写最小缺少条件。只有明确不在本次发布声明内，才可将条件用例记录NOT_APPLICABLE并解释范围。pytest SKIPPED、SDK未安装、fixture返回stub、只运行开发机源码，不是平台PASS。

3. 执行顺序与前置准备

先保留现有工作区与上次证据，在独立QA工作区/测试用户运行。准备合成workspace、受控接收器、专用profile、固定容器/浏览器/MCP/model资产与限定预算。每次故障都要有独立观察者和有限清理步骤，不能让被杀进程自己报告“我已经成功被杀”。

顺序	执行范围	进入条件与结束证据
1	PG01–PG08源码缺口复现、合同与实际命令入口	真实subprocess命令、真实API/socket/session；不能以函数调用代替
2	TC-MAC-001/003/004/005/018/019	真实ARM64、安装资产、路径与Keychain；没有无关权限/联网补包
3	TC-MAC-008/009/010/014/015/016/017	用户服务、OS锁、进程/容器与Artifact回收真实有效
4	TC-INT-001至006、007至012、015	飞书/浏览器/MCP逐个真实能力闭环；缺某一环境只阻塞相应门禁
5	TC-INT-013/014与TC-MAC-022	本地模型来源、实际出口、日志与诊断覆盖全部受管域
6	TC-MAC-011/012/013/021及TC-INT-016	真实睡眠、断网、重启、恢复，再完成产品端到端
7	条件Intel/发行/桌面门禁	只对发布所宣称组合给出通过；真实签名/公证/干净首开另验

设备睡眠、注销、引擎重启等会打断本机工作，应在专用测试设备或明确空闲测试窗口执行，先保存用户其他工作。正常重启不等于物理断电；本计划不以关机断电破坏用户机器来补“强可靠性”结论。

4. 38项用例索引

详细步骤和定位字段以platform_cases.json为准。

ID	用例	优先级	门禁
TC-MAC-001	Apple Silicon 原生身份与通过报告必须来自真实能力	P0	MACOS_ARM64_BASELINE
TC-MAC-002	Intel 原生资格与 Rosetta 结果分开	P1	INTEL_ONLY_IF_CLAIMED
TC-MAC-003	Darwin 控制面与 Linux 容器执行域必须分离声明	P0	MACOS_EXECUTION_BOUNDARY
TC-MAC-004	干净 macOS 用户的真实安装与首次产品启动	P0	MACOS_INSTALL
TC-MAC-005	OFFLINE 干净首用不隐式补下载资产	P0	MACOS_OFFLINE_FIRST_USE
TC-MAC-006	Gatekeeper 真实首开、签名与公证声明一致	P1	DISTRIBUTION_IF_CLAIMED
TC-MAC-007	离线 Gatekeeper 与制品篡改负例	P1	DISTRIBUTION_OFFLINE_IF_CLAIMED
TC-MAC-008	launchd 登录会话中的真实服务注册和环境	P0	MACOS_USER_SERVICE
TC-MAC-009	唯一 profile 锁、父子不自锁与客户端退出	P0	MACOS_SINGLE_INSTANCE
TC-MAC-010	显式停止与卸载仅回收已登记资源	P0	MACOS_STOP_UNINSTALL
TC-MAC-011	真实睡眠/唤醒后的审批、连接与在途任务	P0	MACOS_SLEEP_WAKE
TC-MAC-012	真实断网、网络切换与半开连接后的恢复	P0	MACOS_NETWORK_RECOVERY
TC-MAC-013	注销/登录与受控重启恢复，不承诺登录前常驻	P0	MACOS_LOGIN_REBOOT
TC-MAC-014	真实 SIGKILL、多代/脱离进程与 PID 身份核验	P0	MACOS_PROCESS_RECLAMATION
TC-MAC-015	Docker 客户端死亡、引擎重启与容器独立归属	P0	MACOS_CONTAINER_LIFECYCLE
TC-MAC-016	实际容器权限、出口与资源上限生效	P0	MACOS_CONTAINER_ISOLATION
TC-MAC-017	容器产物 stop/wait→校验→登记→remove	P0	MACOS_ARTIFACT_EXPORT
TC-MAC-018	APFS 路径、大小写/Unicode别名与真实权限	P0	MACOS_FILESYSTEM

ID	用例	优先级	门禁
TC-MAC-019	真实 Keychain 后端、锁定/拒绝与最小注入	P0	MACOS_KEYCHAIN
TC-MAC-020	TCC 按实际功能和执行二进制最小授权	P0	MVP_NO_EXTRA_TCC_AND_OPTIONAL_NATIVE
TC-MAC-021	macOS 数据备份/恢复与版本变更的真实重启	P0	MACOS_RESTORE_AND_UPGRADE_IF_IMPLEMENTED
TC-MAC-022	真实SDK/系统故障诊断不泄漏秘密或启用遥测	P0	MACOS_DIAGNOSTICS
TC-INT-001	飞书真实测试应用接入、配对与 owner 撤销	P0	FEISHU_REAL_ENTRY
TC-INT-002	真实飞书事件持久 ACK、重投去重与重启	P0	FEISHU_DURABLE_INGRESS
TC-INT-003	真实审批卡 callback 的 deadline、重复与失效	P0	FEISHU_APPROVAL_CALLBACK
TC-INT-004	普通卡、CardKit权限不足与三个独立时钟	P0	FEISHU_CARD_DELIVERY
TC-INT-005	飞书发送成功但本地回执丢失的 UNKNOWN 核对	P0	FEISHU_UNKNOWN_SEND
TC-INT-006	真实 SDK 阻塞与进程外 Gateway watchdog	P0	FEISHU_PROCESS_WATCHDOG
TC-INT-007	真实 Chromium 启动、跨进程 profile 锁与沙箱	P0	BROWSER_REAL_EXECUTOR
TC-INT-008	浏览器 Service Worker、重定向与实际出口	P0	BROWSER_NETWORK_BOUNDARY
TC-INT-009	浏览器目标变更与点击成功后断线 UNKNOWN	P0	BROWSER_APPROVED_WRITE
TC-INT-010	MCP 真实 stdio/SDK 握手、环境与流量上限	P0	MCP_REAL_TRANSPORT
TC-INT-011	MCP 工具动态变化、伪造 hints 与旧授权失效	P0	MCP_MANIFEST_CHANGE
TC-INT-012	MCP 写动作超时、取消与副作用核对	P0	MCP_UNKNOWN_WRITE
TC-INT-013	Ollama 本地模型来源、禁云配置与实际出口	P0	OLLAMA_LOCAL_PROVENANCE
TC-INT-014	OFFLINE/LOCAL_MODEL 实际受管域出口和模式切换	P0	REAL_EGRESS_ENFORCEMENT
TC-INT-015	浏览器上传/下载和污染数据进入真实 Artifact 流	P0	BROWSER_DATA_TRANSFER
TC-INT-016	真实 macOS 产品从飞书任务到批准执行、重启和核对的闭环	P0	MACOS_MVP_PRODUCT_E2E

5. 已核实的项目命令及其适用条件

这里列出的是固定提交实际存在的入口；尚未运行。请在完整仓库checkout中执行，不在本次审查的零散repo_snapshot里运行。先核对HEAD与依赖锁，所有测试输出均存入本轮证据，保留上一轮日志而非悄悄覆盖历史。

5.1 基线与默认检查

适用条件：已安装项目要求的Python 3.12系列与uv，允许安装准备阶段联网，仓库锁文件完整。该baseline的pyproject声明Python >=3.12,<3.13；不是本手册猜测的运行时版本。

```
git rev-parse HEAD
git status --short
uv sync --locked
.venv/bin/python scripts/verify.py --phase M0
```

verify目前只接受M0/M1；默认M0，显式把OPENBOT_NATIVE_TESTS从子进程环境删除。它会写evidence/implementation日志/报告，而且没有启动产品服务。不能用不存在的--phase M3，不能把该命令结果当真实macOS/飞书/Browser/MCP验收。

如要按当前脚本检查M1组件准入，命令是：

```
.venv/bin/python scripts/verify.py --phase M1
```

两种phase都不改变“未启动真实服务”的证据边界。失败/timeout必须保留真实退出情况，不能只从最后一行PASS推导全部成功。

5.2 实际CLI可达性

适用条件：在项目完整根目录，安装已经完成。PYTHONPATH仅是当前源码入口诊断方式，不是最终干净安装通过的替代。

```
PYTHONPATH=services/core/src .venv/bin/python -m openbot --help
PYTHONPATH=services/core/src .venv/bin/python -m openbot version
PYTHONPATH=services/core/src .venv/bin/python -m openbot health
PYTHONPATH=services/core/src .venv/bin/python -m openbot task --help
```

最后一条用于无副作用复现主入口是否接受task，不创建真实任务。按当前源码应被主parser拒绝；结果仍须在目标环境用真实subprocess记录。health成功退出也不能证明daemon活着：没有session的当前分支返回固定daemon_running=false与stage=M0。

在修复真实入口以前，本手册不提供虚构的openbot daemon start、doctor、install或可直接用的task create完整命令。修复要提供真实服务启动、认证bootstrap、状态查询与受限transport，并把新命令/help/端到端回归写入对应提交。直接给run_cli_command注入内存session仍只覆盖合同层。

5.3 实际SDK对loopback的合同测试

适用条件：先确认需要的依赖组与锁，允许安装准备阶段联网；测试使用本地合成HTTP，不是真实云账户。

```
uv sync --locked --group provider-openai
.venv/bin/python -m pytest tests/components/contracts -q -rs
```

检查-rs显示的跳过原因和被收集用例；没有装可选SDK而跳过不能记模型适配已测。真正云服务质量、计费和凭据仍是另一门禁，不能从loopback合同反推。

5.4 真实Docker PoC——先审代码和测试后端，再运行

当前真实测试入口确实存在：

```
OPENBOT_NATIVE_TESTS=1 .venv/bin/python -m pytest tests/components/native_poc/test_docker.py::test_actual_native_container_limits_and_independent_lifecycle -q -rs
```

该命令不是当前提交下可跨Mac直接复制执行的推荐命令。 运行前至少完成：

1. 审查docker_poc.py并消除开发者特定的/usr/local/bin/docker及/Users/Zhuanz/.docker/run/docker.sock假设；用明确参数选择经过确认的当前测试用户engine，不能扫描/改用别人socket。
2. 从已锁定来源预置asset文件指向的镜像，实际验证digest、OS/arch，保持pull=never与测试镜像限制。
3. 输出区采用最小可写策略；不把原fixture的0777复制为正式权限修复。具体UID/GID/卷导出方式按Mac后端实测。
4. 确认只创建带独立run_id归属的容器、内存/PID/CPU限制和有限超时；cleanup只操作这些已登记资源。
5. 记录此命令只是原语资格，不替代真实Broker派发、Artifact登记和恢复路径。CPU配置读回不能冒称CPU压力已测，源码PoC本身也不宣称生产executor。

运行成功后仍需TC-MAC-015/016/017与TC-INT-016。未满足前提时留下BLOCKED/待修复信息，不在错误socket上试跑、不拉取latest、不关闭隔离。

6. macOS与外部工具诊断命令

以下命令需在目标Mac执行；本轮没有执行。OPENBOT_QA_*均是由操作者按已核实路径/身份填写的专用变量，不重设HOME等系统变量。占位符不是可原样粘贴的真实路径。任何输出在分享前删除凭据、个人目录和无关进程信息。

6.1 OS、架构和真实进程

```
sw_vers
uname -s
uname -m
sysctl -n hw.optional.arm64
file "$OPENBOT_QA_EXECUTABLE"
```

lipo存在且目标为Mach-O时，可检查lipo -archs。sysctl.proc_translated若存在可诊断当前调用进程的Rosetta状态；键不存在不等于所有进程都已证明原生。还需看Daemon、Python、Rust helper和Chromium各自的实际加载架构。[P01]

```
ps -p "$OPENBOT_QA_PID" -o pid=,ppid=,pgid=,uid=,lstart=,comm=
lsof -nP -a -p "$OPENBOT_QA_PID" -i
```

只传已确定属于测试域的PID。ps时间和PID是诊断资料，不能代替正式Supervisor的信号前创建身份验证，更不能只把同一个字符串传两次证明防PID重用。不要使用会泄漏环境/命令行秘密的全进程转储作为普通诊断包。

6.2 用户服务

适用条件：实际选型是LaunchAgent，目标GUI登录域存在，变量指向本次受测用户和服务。使用SMAppService时还要验证该API的注册/状态和系统登录项许可，不能把以下命令当全部安装实现。[P02][P03]

```
man launchctl
plutil -lint "$OPENBOT_QA_AGENT_PLIST"
plutil -extract Label raw -o - "$OPENBOT_QA_AGENT_PLIST"
launchctl print "gui/$OPENBOT_QA_UID/$OPENBOT_QA_LAUNCH_LABEL"
```

launchctl print可能包含配置或环境；只在私有证据目录保留原始输出，公开报告只摘必要状态/PID/退出码。通过产品正式安装/卸载执行变更；若需要用bootstrap/bootout定位，先读目标OS的man，且仅操作核实的测试label，不对system域或其他用户服务操作。

6.3 Gatekeeper、签名、公证

适用条件：先按用户真实下载/解包/安装方式取得制品，保留隔离属性；工具存在且适用于制品类型。Apple说明Gatekeeper首开结果受缓存影响，干净Mac/macOS VM快照每轮独立最可靠；CLI检查是补充。[P04][P05]

只读诊断：

```
xattr -p com.apple.quarantine "$OPENBOT_QA_DOWNLOAD"
xattr -p com.apple.quarantine "$OPENBOT_QA_APP"
```

```
codesign -dv --verbose=4 "$OPENBOT_QA_APP"
codesign --verify --strict --verbose=4 "$OPENBOT_QA_APP"
```

检查所有实际分发的可执行helper；不能只验外层图标。quarantine的原始属性是诊断信息，不把其字节格式当稳定应用API；程序需要操作属性时遵循Apple的quarantinePropertiesKey文档。[P05]

macOS14+且工具存在的.app诊断：

```
syspolicy_check distribution "$OPENBOT_QA_APP"
```

较旧系统或其他分发形态，依Apple文档选择，不能将三条全部对同一种文件执行：

```
spctl -a -t exec -vvv "$OPENBOT_QA_APP"
spctl -a -t open -vvv --context context:primary-signature "$OPENBOT_QA_DMG"
spctl -a -t install -vvv "$OPENBOT_QA_PKG"
```

适用的.app/.dmg/.pkg安装了Xcode工具时可用xcrun stapler validate验证附着票据；原始CLI二进制不能不加区分地套用stapler。notarization成功仍不替代真实首开与helper加载；离线首开必须在此前没在线信任过该制品的环境完成。[P04][P06]

不执行sudo spctl关闭防护、不删除com.apple.quarantine来“通过”、不使用xattr -dr清除目录属性、不复制chmod 777或重签未知代码作为修复。M3开发试点的签名限制如实记录，M4发行按其声明完整验收。

6.4 Docker实际后端与容器

适用条件：Docker CLI路径、私有QA配置目录和本次engine endpoint已确认。优先显式--config与--host，避免不知情地继承远端DOCKER_HOST/context。

```
"$OPENBOT_QA_DOCKER_BIN" --config "$OPENBOT_QA_DOCKER_CONFIG" --host "$OPENBOT_QA_DOCKER_HOST" version
"$OPENBOT_QA_DOCKER_BIN" --config "$OPENBOT_QA_DOCKER_CONFIG" --host "$OPENBOT_QA_DOCKER_HOST" info --format '
{{.OSType}}/{{.Architecture}}'
"$OPENBOT_QA_DOCKER_BIN" --config "$OPENBOT_QA_DOCKER_CONFIG" --host "$OPENBOT_QA_DOCKER_HOST" container
inspect --format 'id={{.Id}} running={{.State.Running}} status={{.State.Status}} exit={{.State.ExitCode}}' "
$OPENBOT_QA_CONTAINER_ID"
```

可以用选择性inspect检查User、ReadonlyRootfs、NetworkMode、Capabilities、Memory、PidsLimit、NanoCpus、Mount目的地与只读属性，不导出完整Config.Env或个人registry配置。实际context可用docker context inspect选择性读取endpoint，确认后再使用；不要为了让测试通过切换用户默认context。[P09][P10]

回收的验收顺序是产品执行stop仍运行容器→有截止的wait/终态核对→有界暂存导出/路径类型链接权限与配额验证→Artifact登记或失败处置→remove。人为手工清理只能作为测试收尾，不能回填成产品已自动清理。Docker cp支持停止后的容器，-L会跟随源symlink；因此不使用跟随链接的导出方式，更不能把cp成功等同内容验证。[P11]

6.5 真实Ollama本地服务

当前官方文档有两种禁云配置：OLLAMA_NO_CLOUD=1，或server.json中disable_ollama_cloud=true，修改后须重启实际服务。关闭的是云模型/搜索；Mac App自动下载更新是另一条网络路径。必须在受控测试服务作用域验证配置实际生效，不能仅在OpenBot shell设置变量就认为已有Ollama服务继承了它。[P17][P18]

在锁定版本支持、独立测试端口已确认空闲且用户未共享该服务时，可为专用测试实例设置：

```
env OLLAMA_NO_CLOUD=1 OLLAMA_HOST=127.0.0.1:11435 OLLAMA_MODELS="$OPENBOT_QA_MODEL_DIR" "
$OPENBOT_QA_OLLAMA_BIN" serve
```

这是一条持续前台服务命令，应由受管测试进程/harness带独立停止与资源控制启动，不在共享登录会话随意开启第二服务。11435只是示例测试端口，执行前要检查，不能假定固定空闲。模型目录只指向已准备并验明的测试资产；不执行run/pull去补缺模型，不改用户全局launchctl环境。

只读模型清单检查（确认服务监听该地址后）：

```
curl --noproxy '*' --max-time 5 --fail --show-error http://127.0.0.1:11435/api/tags
```

真实推理从产品模型适配器发起，并与所选本地模型digest/进程/出口对照。若用单独API探针，POST /api/generate的model、prompt与上限放合成请求文件，再经受控地址发送；它只证明服务可推理，仍不代表OpenBot的预算/取消/工具循环接通。[P19]

当前Ollama macOS文档写macOS14+、Apple M支持CPU/GPU、x86仅CPU；最终以候选锁定版和实际硬件记录为准。Mac Docker内Linux执行也不能用来证明原生Metal加速。[P18]

6.6 休眠、Keychain和TCC

睡眠/唤醒使用系统菜单与真实设备；注销/重启分别测试。可读取有限时间窗的pmset系统事件帮助定位，但不为测试修改全机电源策略。不要把ClockSnapshot构造、pid=8001或时间戳推进当真实系统睡眠/网络证据。

Keychain只使用独立测试用户/合成secret，记录实际keyring后端与应用签名身份。通过测试钥匙串/系统UI锁定和拒绝访问，不重置默认钥匙串、不把secret放security命令行参数、不运行会把secret打印到stdout的读取命令。Apple说明登录/睡眠/密码状态会影响钥匙串解锁，必须在launchd实际用户会话验证。[P20]

ScreenCapture/Accessibility采用实际helper的预检与系统UI授权/撤销；MVP headless无需这两项。Apple的AXIsProcessTrustedWithOptions判断当前进程，不能从UI壳得到True就给另一个helper授权。文件夹权限、录屏、辅助功能各自验证，拒绝一个不应自动索取Full Disk Access。[P21][P22][P23]

7. 真实集成的判定细节

飞书

官方长连接针对企业自建应用，多个client竞争消费而非广播；事件处理有3秒响应和超时重推规则。同步卡片callback同样有3秒回应要求，但应按其独立回调合同处理，不能复用事件重推假设。[P12][P13]

最小产品证据链是：真实测试用户输入→SDK收到真实event→inbox已提交→Task/Operation→具体审批→真实执行receipt→Outbox→真实message/card。API200、CardKit sequence小实验或WebSocket101握手是有用连通/协议资格，不能独自证明这条链。

callback token的30分钟/两次示例可在官方具体组件文档见到；它不是所有CardKit/消息PATCH的通用期限。要先产生callback，再测试它自己的token；业务approval TTL与消息生命周期分别记录。本轮不引入未经目标API核实的通用14天或30天逻辑。[P14]

平台没有公开测试故障开关时，可在自有adapter/受控receiver边界断开响应或注入错误，并明确哪段真实、哪段合成。不得为制造429向真实租户大量施压；真实消息只发送到明确获准的测试收件范围。

Playwright

官方BrowserType说明chromium_sandbox缺省false；服务工作线程缺省allow，且context.route/page.route不能捕获被Service Worker截获的请求。[P15][P16]

因此实际用例必须启动受管Chromium、记录有效启动状态，测试SW与独立出口边界，而不是通过参数字典/默认lease字段。专用profile只是会话隔离，MCP/Browser名称本身也不是沙箱。跨进程profile互斥要按目录身份验证，不能只靠内存profile_name字典。

MCP

按锁定SDK/协议版本发送真实消息并核验初始化；定义变化通过真实通知触发冻结。annotations不具有权限权威；list_changed/hash只能约束被观察到的定义，不能证明服务器实现代码和运行行为安全。[P24]

响应上限必须约束读入期间的内存/时间/帧；对已经完整分配到内存的bytes再截断不能证明有界。超时后不能把截断JSON或没有receipt当成功，也不能将取消通知/进程结束等同写副作用回滚。

云 Provider 与真实 Ollama 分开验收

真实云 Provider、官方SDK对loopback的合同测试、真实Ollama本地推理是三个独立结果。TC-INT-016按所宣称的Provider组合分别留下结果：选择云Provider时，应使用获准测试账户、合成任务、硬预算和真实产品适配器，记录实际模型/请求ID、用量、工具循环以及取消/超时后的结果；SDK本地故障服务器通过不能替代这次真实服务测试。选择Ollama时，另执行TC-INT-013/014验证模型资产、禁云配置与实际出口，不能拿一个云模型成功响应作为本地模式通过证据。

若本次Mac试点明确仅支持一种已验证组合，其他组合可暂不纳入发布声明；如果README或MVP声明同时支持云Provider与Ollama，两者都要各自通过。缺云凭据只阻塞相应真实云门禁；不阻塞已经具备环境的本地工程复现，也不能把未接通的本地产品入口归类为凭据缺失。

网络观测

“未见请求”必须写观察范围：OpenBot自身、Gateway、Worker、browser、native helper、Docker backend/VM和Ollama可能不在同一进程/网络域。仅mock httpx无法覆盖requests、DNS、原生扩展或子进程。

至少先用获准canary目的地证明监测链工作，再按模式验证允许/禁止路径。记录IPv4/IPv6、DNS、代理与未覆盖协议；没有足够平台边界时就把强离线保证标未验证，不能临时放行外网掩盖问题。LOCAL_MODEL限制模型路径不等于自动禁用飞书，其他通道仍按其policy明示；OFFLINE则不应启用互联网飞书/浏览能力。

8. 与核心状态/恢复用例衔接

平台用例使用真实OS/SDK/服务观测；核心用例检验权威状态，不重复发明另一套恢复语义。

平台用例	需联合核验的核心场景
TC-MAC-014、TC-INT-005/009/012	TC-CORE-015–018：claim/dispatch/外部成功/结果提交四类真实崩溃窗口
TC-MAC-009/014/015、TC-INT-016	TC-CORE-019–024：在途取消、去重、安全重试、旧epoch、UNKNOWN quarantine、新ID绕过与核对证据
TC-MAC-021	TC-CORE-028–031：限量测试卷满盘、外部成功后满盘、在线备份、缺失Artifact/secret_ref恢复
TC-INT-013/014	TC-CORE-036：模式、代理与首用出口；本文件提供真实Ollama与受管域证据
TC-MAC-019/022	TC-CORE-037/038：最小秘密注入、钥匙串失效、跨片段脱敏与扫描器不联网
SDK/模型/网络相关	TC-CORE-039/042：实际请求数、唯一重试责任、预算与取消/断流计费未知

故障注入必须可证明已达到目标持久化边界，再由外部控制器发故障；在同一个对象里直接调用recover方法，不构成新进程恢复证据。效果次数由独立持久计数器/真实服务receipt获得，不能在测试里手填0或1再断言正确。

9. 失败定位、修复与提交的工作方式

按单个可复现问题推进，保留失败证据后再修复；一个问题一个独立commit并push，以用户要求为准，不把多个无关问题一次提交。Git操作细节由配套Git闭环手册执行，本文件不做远端写入。

每个问题记录：

1. 受测commit/build、用例与真实环境、实际命令和退出码、最小输入。
2. 首次偏离发生在哪个边界：入口/认证→真实transport→应用用例→writer→Broker→SDK/OS→独立效果→receipt。
3. 失败前后的状态、真实PID/container/message ID和脱敏证据hash。

4. 原因与最小修复；不能删除失败测试、把断言放宽、关闭policy、清空DB/审批或降低隔离。

5. 修复后的同一负例、一个正常例和关联回归；commit SHA与push成功的远端SHA。

6. 本问题修复并push后再开始下一问题；若push失败保留commit与明确错误，不记远端已完成。

不要把某次原生用例不通过一律归因为“缺环境”。例如CLI parser拒绝task、installer只写JSON、MCP没有transport是实现/接线问题；没有Intel实机、专用飞书测试应用或发布证书才是相应外部条件缺失。

10. 证据与退出条件

每次真实用例至少记录：case_id、commit、build hash、执行UTC时间、OS build/arch、用户会话类型、SDK/engine/browser/model版本与资产digest、具体步骤/命令、预期与实际、真实退出/响应、关联IDs、原始日志hash、被观察范围、剩余限制和状态。

不要把凭据/完整环境、cookie、授权nonce、个人源码/聊天、原始截图或未脱敏网络载荷提交Git。用合成canary和最小元数据；原始敏感取证保留在受保护位置，提交去敏证据与摘要。

macOS MVP通过须同时满足：

- 当前公开的产品入口可真正创建/查询/批准/取消任务，跨进程访问权威Daemon。
- 该Mac与声明后端的P0用例通过，实际生命周期/容器/Keychain/Browser/MCP/飞书能力没有用stub替代。
- UNKNOWN、取消与旧epoch的核心门禁和真实副作用计数一致，恢复不盲重发。
- 真实飞书、模型或发布资产缺环境时分别列BLOCKED，不将未验能力写成可用；若它属于所声明MVP必备能力，则该版本MVP结论仍未通过。
- Intel、Windows、Linux、M4桌面及发行能力按实际范围单列；不因Mac本地通过改写三平台完成状态。
- 每项已修复问题有独立commit、push记录和相关回归；存在未解决P0时不宣称严格MVP验收完成。

11. 官方资料核验索引

本轮于2026-11-05访问。以下是平台合同/文档证据，不是OpenBot在目标环境的通过证据；版本与发布资产仍需锁定。未依赖第三方博客或未经复现的历史issue来设定统一性能指标。

编号	官方资料	已核实事实与使用边界
P01	Apple universal macOS binary、Porting to Apple silicon	universal/原生/Rosetta区分；所有helper与本地库也需架构验证
P02	Apple Service Management	LoginItem/LaunchAgent在用户会话中运行，不能用高权限系统服务替代用户agent
P03	Apple Creating Launch Daemons and Agents、SMAppService	用户agent与登录生命周期；旧归档命令不能替代目标OS的man和实际API状态
P04	Apple DTS Testing a Notarised Product	干净机器/快照、quarantine真实下载、按用户方式首开；可选离线测试票据；按制品类型选择诊断
P05	Apple DTS Resolving Trusted Execution Problems	quarantine传播、Gatekeeper缓存、syspolicy_check及签名/加载问题区别
P06	Apple notarization、macOS14 release notes	公证要求与工具适用条件；syspolicy_check从14提供，不把工具成功当全部首开验证

编号	官方资料	已核实事实与使用边界
P07	Docker Mac permission requirements	Docker engine/containers运行于Linux VM；宿主bind mount权限仍适用
P08	Docker platform FAQ、Desktop settings	宿主共享目录与容器bind mount范围；ECI/VM/后端能力需逐组合核验
P09	Docker context inspect	可读取实际context endpoint；不应盲继承用户默认上下文
P10	Docker container inspect	可按字段读取实际容器状态；不依赖客户端进程退出推断容器结果
P11	Docker container cp	可从已停止容器复制；权限/所有权有方向和参数差异，-L跟随symlink
P12	飞书长连接事件接收	企业自建应用、事件3秒响应/超时重投、client竞争消费
P13	飞书接收callback、callback概述	实际card.action.trigger与3秒响应；动态页面正文可见度有限，重推语义须按选定版本再次确认并保留原文/真实记录
P14	飞书交互组件token示例	具体callback token示例30分钟/两次；不是所有卡片API的通用期限
P15	Playwright BrowserType	persistent profile、Chromium sandbox与env缺省等需显式配置
P16	Playwright BrowserContext、Service Workers、Network	route与SW的覆盖边界、service_workers=block；路由不等于OS网络隔离
P17	Ollama FAQ、官方envconfig源码	禁云参数及重启要求；loopback不证明本地推理，Mac App还有更新网络行为
P18	Ollama macOS	当前OS/CPU/GPU要求与文件位置；须按锁定版本实测
P19	Ollama list models API、generate API	实际模型清单与生成接口；示例模型名不作为项目默认模型
P20	Apple Keychain自动锁定、Keychain登录密码状态	登录/睡眠/密码状态影响解锁；本计划不重置个人钥匙串
P21	Apple录屏权限、Privacy & Security	用户可逐功能授予/撤销，不是全权限先决条件
P22	AXIsProcessTrustedWithOptions、CGPreflightScreenCaptureAccess	权限以实际执行进程与API结果为准，不推断永久授权
P23	Apple APFS格式、控制共享权限	大小写敏感与普通APFS分别测试；文件夹/TCC权限具体化
P24	MCP tools规范、当前官方规范源码	工具提示不可信；固定协议/SDK组合，动态工具变化不得提升授权

本轮未实测与未确定项

没有用户Mac的实际OS build、硬件、已启动服务、签名证书、Keychain/TCC状态、Docker endpoint、浏览器/MCP资产、Ollama模型或飞书测试会话执行结果。本轮也没有验证任何新commit或push，未测物理断电、Intel实机和其他平台。动态官方文档或main源码的现状不能覆盖锁定发行版本；执行前要再核对对应版本。环境实际就绪后逐用例留下证据，不从本手册生成虚假的PASS。

08 OpenBot：逐问题修复、提交与推送规程

版本：2026-11-05。用途：交给用户本机的 Codex / Claude Code 执行。

本轮工作是验收现有实现和逐项修复，不重新生成项目。用户已授权每修复一个问题就 commit、push 一次，不需要逐次再询问。 仓库中的一般性“未经授权不推送”约定已满足授权前提；其他代码、测试、安全和分支规则继续遵守，不改写 AGENTS.md 来制造永久授权。

已确认项目为 xiongcp/OpenBot。本次只读审查参考 main 的 a223d36a659b7ecf32fca2b4890fc187d3e96e48；它不是用户本机当前 HEAD，也不是已经通过验收的版本。开始执行时记录实际本机基线。建议使用 mvp/macos-hardening 或明确的现有开发分支，不自动向 main 提交或推送，不自动 merge、打发布 tag 或部署。

本规程制定期间未在用户 Mac 执行测试、修复、commit 或 push。模板里的执行结果都是 NOT_RUN，待执行 SHA 为 null。

1. 不可跳过的约束

1. 一个独立根因对应一个原子修复提交，包含该问题的最小代码变更、有意义的回归和必要说明。一个根因引发多个错误可以一起修；不同根因不能因为同在一个文件而合并。
2. 修复顺序为：复现 → 锁定失败回归 → 最小修复 → 针对及相关回归 → 明确文件暂存 → commit → 核验实际提交及干净 checkout → push 明确目标 → 核实远端 → 对精确 SHA 核验 CI → 才能关闭问题。
3. 同一仓库只指定一个集成人员/代理操作 index、commit、push。其他代理可并行审查或运行隔离的只读复现，不争抢工作树/暂存区。
4. **最多一个尚未确认推送的修复候选。** push 失败、鉴权阻塞或结果未知时，先处理这一项；可继续盘点、独立复现、补证据，不能继续形成第二项代码修复或未推送提交。
5. 已推送提交不 amend、不 rebase、不 squash 重写。后续回归使用新的修复提交或明确的 revert 提交，再单独验证和 push。禁止 --force、--force-with-lease、+refspec、--mirror、--all、绕过 hooks/保护分支。
6. 用户现有 staged、unstaged、untracked 文件和未推送提交均先识别、保留。不得用 git add .、git add -A、commit -a 混入；不得用 reset --hard、clean、自动 stash 或覆盖文件来获得“干净状态”。
7. 缺真实服务凭据或原生环境只阻塞相应门禁，不妨碍其他独立工作；BLOCKED、SKIPPED、未收集到测试及 NOT_RUN 都不等于通过。

2. 首次绑定仓库、分支和基线

项目地址已经明确，不再泛泛询问“仓库在哪里”。本机根据用户给出的目录、当前 cwd 与真实 Git 配置定位工作副本。只有发现多个相互冲突的工作副本、origin 指向未确认项目/分叉、多个 push URL，或目标分支无法判定时，才问这个具体差异。

在任何写操作前记录：

- 仓库绝对路径、Git 版本、当前分支或 detached 状态、HEAD commit/tree、上游分支。

- 实际 fetch URL 和 **push URL**、目标 refs/heads/...、远端目标是否存在及其 SHA、保护规则与 CI 触发条件。

- dirty 状态、已有暂存内容、相关未跟踪文件、子模块状态及已有未推送提交的归属。

- 当前源码与锁文件、Python/uv、macOS 版本和 CPU 架构。Rosetta 与原生运行分别记录。

以下是 Bash 模板，不是已经执行的命令；变量必须来自本机绑定记录。raw_dir 必须在仓库之外，使用新建的当前用户私有目录，或先验证已有目录属于当前用户、权限为 0700 且不是符号链接；umask 不会修复旧目录权限。不要把带凭据的 URL 或原始 diff 输出到聊天或公开日志。

```
: "${repo_dir:?先绑定真实仓库绝对路径}"
: "${raw_dir:?先绑定仓库外私有证据目录}"
umask 077
mkdir -p "$raw_dir"

git -C "$repo_dir" rev-parse --show-toplevel > "$raw_dir/repo-root.txt"
git -C "$repo_dir" rev-parse --verify 'HEAD^{commit}' > "$raw_dir/initial-head.txt"
git -C "$repo_dir" rev-parse --verify 'HEAD^{tree}' > "$raw_dir/initial-tree.txt"
git -C "$repo_dir" symbolic-ref --quiet --short HEAD > "$raw_dir/initial-branch.txt"
git -C "$repo_dir" status --porcelain=v1 -z --untracked-files=all --ignore-submodules=none > "$raw_dir/initial-status.nul"
git -C "$repo_dir" diff --no-ext-diff --no-textconv --binary > "$raw_dir/initial-unstaged.patch"
git -C "$repo_dir" diff --cached --no-ext-diff --no-textconv --binary > "$raw_dir/initial-staged.patch"
git -C "$repo_dir" worktree list --porcelain > "$raw_dir/worktrees.txt"
```

symbolic-ref 返回非零可能表示 detached HEAD，不应吞掉后继续猜分支。逐条保存退出码。NUL 分隔的状态文件用解析器处理，不按空格拆文件名。[G02][G03]

origin 只是优先检查的名称，不是已验证推送目的地：

```
: "${repo_remote:?从真实配置确认remote名称，通常为origin}"
git -C "$repo_dir" remote get-url --all "$repo_remote" > "$raw_dir/fetch-urls.txt"
git -C "$repo_dir" remote get-url --push --all "$repo_remote" > "$raw_dir/push-urls.txt"
```

get-url --push --all 会显示实际 push URL，并展开 Git URL 重写配置。确认只有一个已授权目的地；如果 fetch/push URL 不同，后续远端核验必须查 **push 端点**，不能用 fetch 端点误作成功回执。URL 含口令/token 时，不将其放入命令参数或证据摘要，改用现有安全凭据机制；不能擅自改远端或认证权限。[G04]

dirty worktree 的默认处理

优先在确认的 commit 基线上建立独立工作树，用户原目录与原 index 保持不动。工作树有自己的 HEAD/index，但共享仓库对象和部分配置，不应当作与原仓库完全隔离的安全沙箱；创建时仍遵守已审查的 hooks。[G03]

```
: "${baseline_sha:?填实际确认的起点commit，不照抄参考快照}"
: "${repair_branch:?填确认的修复分支}"
: "${repair_dir:?填不存在或空的独立工作目录}"
git -C "$repo_dir" check-ref-format --branch "$repair_branch"
git -C "$repo_dir" worktree add -b "$repair_branch" "$repair_dir" "$baseline_sha"
```

分支已存在时先检查归属和是否已被另一工作树使用，不使用 -B 或 --force 覆盖。若修复依赖用户未提交改动，不声称新工作树包含它们；先指出基线差异，继续可独立执行的检查。不能把用户全部未提交内容搬进自己的修复提交。

如果拟推送范围包含归属/发布授权不明的既存未推送提交，停止该 push 范围，先明确这些提交是否属于本次授权；不能靠只写了一个新 commit 就假称只发布一项修复。

3. 从真实仓库发现测试命令

首先读当前 checkout 的 AGENTS.md、README、pyproject/锁文件、scripts/verify.py、测试目录和顶层 CI 配置，形成 command_manifest。每项记实际 argv、cwd、环境键名、依赖组、代码来源、超时、测试层级和外部副作用；不要用 eval 执行仓库文本。

本次参考 SHA 已存在这些入口，**本机应先复核，再运行**：

```
cd "$repair_dir"
uv sync --locked
.venv/bin/python scripts/verify.py
uv run --no-sync pytest --collect-only -q
uv run --no-sync pytest
```

前两个来自 README；后两个用于复现当前 CI 使用的 pytest console 入口与实际 collection。不能用另一个入口的绿灯掩盖 CI 命令的导入错误。[R01][R02]

参考版本的 scripts/verify.py 会：

- 设置 core 与 feishu_gateway 的 PYTHONPATH，内部以 python -m pytest 运行。
- 删除 OPENBOT_NATIVE_TESTS；默认 phase 为 M0，只接受 M0/M1。
- 将原始日志和 JSON 写到 evidence/implementation；每条检查有 60 秒 timeout。

因此它通过不证明原生 Mac、真实服务或 M3 已验收。原始输出先存私有证据，不因位于仓库内就全部暂存；在自己独立工作树中检查并单独管理生成文件，不能覆盖用户原目录中的证据。[R03]

pyproject.toml 的默认组为 dev-contracts，provider-openai 为额外组。需要 SDK 合同测试时，先核对当前组名和锁，再按现有组安装，例如该参考版本可用 uv sync --locked --group provider-openai。缺 SDK 导致 skip 不能代替对应测试通过。真实凭据只在专用测试环境注入，不写入命令行、Git、模型或日志。[R04]

不臆造 openbot doctor、openbot acceptance、--profile 等接口。需要原生启动、profile、后台服务或真实模型命令时，必须从当前 CLI/parser、脚本或测试代码找到可用入口；没有入口就登记具体缺口。

4. 每个问题的执行闭环

4.1 建立问题记录并先复现

记录 issue_id、严重度 S0—S3、相关测试优先级 P0—P2、当前 base SHA、受影响能力、期望/实际、最小输入、命令、真实/模拟边界和证据。一个失败首先分类为产品、测试入口、CI、环境/凭据、测试本身或尚未解释，不能将所有失败都归因环境。

运行基线用例证明问题确实存在。新增或扩展一个能锁住根因的失败回归；如果已有测试可稳定复现，保留该测试并将其纳入回归清单，不复制无意义断言。纯文案/CI 配置类问题使用有意义的配置/触发/实际运行证据，不编写只镜像改动文本的测试。

回归要证明：修复前确因该缺陷失败，修复后因正确行为通过；测试必须被收集且实际执行。导入失败不能充当业务断言失败，未触发的 workflow 不能算 CI 通过。竞态用屏障/事件确认时序，覆盖两种线性化次序，不靠 sleep 碰运气。

4.2 最小修复与局部验证

仅修本问题根因，不附带格式化全仓、重构、升级不相关依赖或另一个 bug。新增必要生产路径回归，不让测试只跑一份替代实现或参考 DDL。

完成针对测试、具体受影响回归和仓库要求的 lint/type/format。对授权、持久化、恢复、隔离、数据出口的修改，包含相关 P0 反例；把以前修复的相关回归加入当前集合。不能删断言、添加无理由 skip、放宽权限、catch-all 返回成功或伪造结果字段。

真实环境缺失时记录 BLOCKED_ENV/BLOCKED_CREDENTIALS 与所缺条件，可继续不依赖它的测试。未执行的真实门禁仍未通过；本地模拟通过不能补足它。

若完整基线已被另一独立问题阻塞，保存该失败的基线证据和问题关联；本问题针对验证与可运行的相关检查仍必须通过，且不能有新增回归。可以据此提交并 push 已验证的局部修复，但状态保留 LOCAL_VERIFIED/CI_BLOCKED 等相应范围，直到所需验证齐全才 FIXED。只能由 CI 验真的配置变更，以本地有意义校验作为 push 前提，以实际 CI 结果作为关闭前提。不得将自己新引入的失败称为历史问题。

4.3 审查并明确暂存文件

在集成人员的工作树中确认没有别人的 index 内容。形成 approved_paths，它是具体相对文件列表，不是目录、glob 或全仓。检查实现、测试、生成文件和必要文档是否都属于同一问题，检查依赖锁改变是否确有原因。

```
: "${repair_dir:?}"
: "${raw_dir:?}"
git -C "$repair_dir" diff --cached --quiet
# 若返回1表示index已有变更：先核对归属，不能盲目继续。

# fix_paths由本问题已审查的具体相对文件填入；不使用占位文件名。
((${#fix_paths[@]} > 0)) || exit 1
git --literal-pathspecs -C "$repair_dir" add -- "${fix_paths[@]}"
git -C "$repair_dir" diff --cached --check
git -C "$repair_dir" diff --cached --name-status -z > "$raw_dir/staged-paths.nul"
git -C "$repair_dir" diff --cached --no-ext-diff --no-textconv > "$raw_dir/staged.patch"
tested_index_tree=$(git -C "$repair_dir" write-tree)
```

将 staged 文件列表与 approved_paths 精确比较，核验没有用户改动、原始日志、秘密、PII、模型缓存、浏览器 profile、用户 DB 或无关构建产物。秘密扫描必须离线且审查输出，另做人工/规则审查；扫描阴性不是公开许可。git write-tree 只给出 index 树，需要另查工作树与测试输入一致性。[G05][G06]

4.4 单问题 commit 与 hooks

提交信息遵循仓库约定，至少包括问题 ID、根因/行为变化、测试命令及范围，使用现有作者/签名身份，不伪造身份，不加无依据的工具归属。

```
: "${commit_message_file:?私有目录中的已审查提交说明文件}"
expected_parent=$(git -C "$repair_dir" rev-parse --verify 'HEAD^{commit}')
git -C "$repair_dir" commit -F "$commit_message_file"
fix_sha=$(git -C "$repair_dir" rev-parse --verify 'HEAD^{commit}')
fix_tree=$(git -C "$repair_dir" rev-parse --verify 'HEAD^{tree}')
git -C "$repair_dir" show -s --format='%H%n%T%n%P' "$fix_sha" > "$raw_dir/commit-object.txt"
git -C "$repair_dir" diff-tree -r --no-commit-id --name-status -z "$expected_parent" "$fix_sha" > "$raw_dir/committed-paths.nul"
```

逐条检查退出码；commit 返回异常也要查询实际 HEAD，不能盲重试。核对最终提交只有预期父提交、只含这一问题、文件范围正确、内容对应已验证树。hook 修改 index/文件可能使 fix_tree != tested_index_tree；此时先审查实际差异，再重测最终提交，不能把 hook 当自动批准。[G07][G08]

pre-commit 失败而 HEAD 没变时，检查它改了哪些文件，仅暂存本问题的合法变动并重新测试后再提交。不要用 --no-verify 或改 hooksPath 绕过。若提交已生成但内容不合格，保持本问题，保存未发布候选并构建正确替代候选；没有验证通过前不 push，也不开始另一问题。

4.5 验证实际 commit，而不只验证工作目录

先检查最终 commit 的 parent/tree/diff、必要测试是否随提交进入 Git、未跟踪内容是否意外参与先前测试。再用新建的干净 detached 工作树运行针对与相关验证，记录**实际 checkout SHA**：

```
: "${verify_dir:?本问题新建的独立验证目录}"
git -C "$repo_dir" worktree add --detach "$verify_dir" "$fix_sha"
git -C "$verify_dir" rev-parse --verify 'HEAD^{commit}' > "$raw_dir/verified-checkout-sha.txt"
cd "$verify_dir"
```

```
# 按command_manifest安装实际需要的已锁依赖组，并运行实际命令。
uv sync --locked
.venv/bin/python scripts/verify.py
```

在启动测试前确认工作树仍对应该 commit，hooks未改变源码/测试/锁/配置；有偏差先查明，不把脏副本记作干净验证。

以上基线命令之外仍需该问题的目标/相关测试；涉及 provider-openai 则在本次干净环境安装其锁定组，不能因默认组缺包而把回归跳过。记录 tests collected/executed/passed/failed/skipped、每个 skip 原因和每条退出码，避免只看一行 PASS。

验证脚本生成的文件先按已审查清单保存私有证据；源文件、测试、锁或配置发生意外变动则证据失效。仅可在自己拥有的临时工作树中处理自己生成的文件，原用户工作区不动。后续任何更改都需重新确认测试对象。

4.6 push 精确目标

确认 fix_sha 已通过实际提交树验证，当前修复分支未被其他人推进；再次核对真实 push endpoint、目标 ref 与远端变化。target_ref 必须为完整 refs/heads/...。

```
: "${target_branch:?确认的远端修复分支，不能猜main}"
target_ref="refs/heads/$target_branch"
git -C "$repair_dir" check-ref-format "$target_ref"

# push_endpoint是已核对的单一实际推送端点，不含口令/token。
git -C "$repair_dir" ls-remote --exit-code --refs "$push_endpoint" "$target_ref" > "$raw_dir/remote-before.txt"
```

ls-remote --exit-code 的 2 表示没有匹配 ref，不等于鉴权失败；创建新修复分支必须是已经确认的动作。其他错误保持阻塞。不要把旧的本地 origin/... 缓存当实时远端。[G09]

明确 SHA:ref 推送，禁止附带 tags、mirror 或对子模块自动推送。已有子模块提交必须在其正确远端可达；否则单独解决，不暗中发布其他仓库。

```
if git -C "$repair_dir" \
 -c push.followTags=false \
 -c push.recurseSubmodules=check \
 -c "remote.${repo_remote}.mirror=false" \
 push --porcelain "$repo_remote" "${fix_sha}:${target_ref}" \
 > "$raw_dir/push.stdout" 2> "$raw_dir/push.stderr"
then
 push_exit=0
else
 push_exit=$?
fi
printf '%s\n' "$push_exit" > "$raw_dir/push.exit"
```

无论返回 0、超时、断连还是非零，都重新查询实际 push 端点对应 ref；推送意图不能代替交付回执。[G01]

```
git -C "$repair_dir" ls-remote --exit-code --refs "$push_endpoint" "$target_ref" > "$raw_dir/remote-after.txt"
```

4.7 验证远端及处理不确定结果

观察	状态及动作
远端目标 SHA 等于 fix_sha	PUSH_VERIFIED，即使客户端曾超时也不再新建或重复提交；记录回执时间/端点/ref
远端比 fix_sha 更新，且能证明包含 fix_sha	记录 REACHABLE_NOT_TIP：修复已可达，当前分支新 tip 尚未自动获得旧验证；选择候选重验
远端仍为原 SHA，确定没有包含本次修复	同一 SHA 的有界、非 force 重试；不创建第二修复提交
查询失败、鉴权不明或不能确认包含关系	PUSH_UNKNOWN / PUSH_BLOCKED，不猜成功或失败；不积累下一项未推送修复
non-fast-forward	读取最新目标、分析新增提交；只在未发布修复上构建新基线候选，重测后正常 push，绝不 force

确认包含关系时 fetch **实际 push 端点** 到本次专用本地 ref，而不改用户分支。fetch_ref 使用经过 check-ref-format 校验的唯一会话名，避免覆盖他人的追踪 ref：

```
: "${fetch_ref:?本次专用且尚不存在的本地ref}"
git -C "$repair_dir" check-ref-format "$fetch_ref"
git -C "$repair_dir" fetch --no-tags --no-recurse-submodules "$push_endpoint" "${target_ref}:${fetch_ref}"
remote_tip=$(git -C "$repair_dir" rev-parse --verify "${fetch_ref}^{commit}")
git -C "$repair_dir" merge-base --is-ancestor "$fix_sha" "$remote_tip"
```

ancestor 判定返回 0 为真、1 为假，其他非零为错误，不能一律当“未包含”。[G10][G11]

non-fast-forward 且本修复确未发布时，可在新的独立工作树从已核对 remote_tip 出发，将**本问题唯一修复**用 cherry-pick --no-commit 应用，解决可解释冲突，重走测试/暂存/commit。旧未发布 SHA 标 SUPERSEDED_LOCAL，只保留一个活动候选，不把旧候选一并 push。已发布或发布未知的提交不走这种重写流程。[G12]

4.8 CI 对准实际代码并关闭问题

在 CI 服务查询目标分支和明确的 commit，核对 workflow 路径、事件、branch/path filters、run/attempt/job、实际 checkout SHA、依赖组、runner OS/arch、结果和日志。只存在 YAML、run 状态 completed 或绿色 badge 都不足够。

PR 工作流可能默认测试合并合成提交。记录 PR head、run 元数据 SHA 和**实际 checkout SHA**，以真正测试的代码为准；不能把默认分支的结果嫁接到修复 SHA。定时/手动触发也要核对执行的是哪个 ref。[G13][G14]

FIXED 必须满足：确定性复现、有效回归、目标/相关验证通过、合法原子提交、远端已确认、针对同一 verified_on_sha 的所需 CI/真实门禁通过。introduced_by_sha 是引入修复的提交，verified_on_sha 是实际完成验证的提交，通常相等；若后续修复了独立 CI 基线阻塞，可在包含该修复的后续候选上重新验证，明确记录祖先关系和全部相关结果。

避免多根因 CI 死锁： 上一修复已经 push 且远端确认，但 CI 被另一项已识别独立基线故障阻塞时，保留 CI_BLOCKED，不标 FIXED；可以继续修那个独立 CI 根因，每项仍单独 commit/push。本规则不允许在 push 未确认时堆积修复，也不把 CI_BLOCKED 当通过。最终候选必须清空必需门禁阻塞。

5. 已知只读基线的处理顺序

参考版本的仓库/CI审查发现了这些线索，**本机须重新核对，它们不是本规程已执行的修复结果**：

- .github/workflows/pr.yml 由 push/pull_request 触发，三 OS matrix，固定 Python 3.12.12/uv 0.9.26；实际测试命令为 uv run --no-sync pytest。
- 参考 CI run 41949551726 失败：Mac/Windows 在解释器安装，Linux 有 24 个 No module named tests collection 错误，另有 SDK skip。这些应按实际根因分别复现、修复，不能用“本机环境不同”忽略 collection。

- platform-macos.yml 仅 PR/workflow_dispatch，直接运行 python3/pytest，未见依赖安装步骤；普通 push 不等于此工作流运行。
- 放在 .github/workflows 子目录中的 nightly/soak、pull_request/gate、release/supply_chain 文件不能视为已被 GitHub 识别的工作流；GitHub不支持 workflow 子目录。[G15]
- runner/解释器可用性以执行时官方支持矩阵为准；不要继续把失效 label 或缺少对应架构构建的精确版本当可运行组合。固定实际 OS image/arch 与 runtime 后重新验证。

优先恢复可运行的测试入口和 CI。runner label、解释器构建、导入根因、依赖安装和 workflow 未发现应按独立根因建记录，不能整包修改后只推一个“大修复”。

6. hooks、push 与回归异常决策

异常	处理
pre-commit 改文件并失败	看 HEAD 是否变化；复核 hook 改动，明确暂存本问题内容，重新跑受影响测试；不 no-verify
hook 成功但实际 commit tree 改变	查实际 commit 范围并对干净 checkout 重测；未确认前不 push
commit 后本地验证失败	保留未发布证据，只继续本问题；构建合格替代候选，不推失败提交、不接下一修复
push 返回超时/断连	先实时查目标 ref；已到达就确认，不重新 commit；不能查清就 PUSH_UNKNOWN
auth/签名/保护分支阻塞	使用既有合法认证/签名；必要登录由用户本机完成，不索要明文token；不关闭保护、不偷偷换目的地
已指定目标禁止直接 push	遵循已经明确的修复分支/PR流程；若需要改变明确指定的目标，说明具体冲突后再确认一次，不泛问每项是否能 push
远端前进	判断是否包含修复，或在新基线上重建未发布同一问题候选；重新测试，禁 force
CI pending/未触发	明确等待/阻塞原因，核对触发条件；不凭仓库存在 YAML 标 PASS
后续修复打破先前功能	记录新的回归问题并使相应旧结论 REOPENED/失效；新 commit+push，不 amend 已推送历史
失败看似 flaky	保留失败样本并定位时序/环境原因；不能只重跑到绿、删除断言或扩大timeout掩盖
暂存区/commit 含秘密	未发布时禁止 push，不用“下一提交删除”掩盖泄漏历史；重建干净候选。若已发布，升级S0并按授权应急流程处理，普通流程不擅自强推清历史

7. 证据、自引用与隐私

分两层保存：

- **私有原始证据**：仓库外受限目录中的完整 stdout/stderr、真实请求/回执、必要数据库快照、原生进程/容器观测、未脱敏截图。有限保留、最少采集，不进入 Git/聊天/公开 CI artifact。
- **可入 Git 的摘要**：合成 fixture、相对源码路径、用例 ID、命令及安全参数、版本/OS/arch、计数、预期/实际和脱敏结果。移除 tokens、headers/cookies、个人路径、姓名/邮件、tenant/user IDs、用户文件内容、截图可见隐私及带签名查询参数的 URL。

可审查摘要引用不透明证据 ID；私有账本再把 ID 映射到原始文件。不要把秘密的哈希或真实私有绝对路径当作安全公开替代品。扫描必须离线；外部内容的 allowlist 注释不改变脱敏/出口要求。

不要求修复 commit 自己包含自己的 SHA。 同一提交可放 issue_id、base_sha、run_id 和脱敏结果；若放测试树 hash，必须注明它指向哪个已测快照，不把加入摘要前的 tree 当成最终 commit tree。最终 commit/tree、push、CI 回执写

到 Git 外的运行账本。禁止为了补自身 SHA 不断 amend 或为每个 bug 再推一个纯记账提交。

最终汇总如需入 Git，作为独立且明确的文档变更提交，更新候选 SHA 并跑相应门禁；也可作为仓库外、绑定固定候选 SHA 的交付报告。

8. 每项汇报的最小内容

issue_id / S级 / 关联P级用例；复现与根因；改动文件；失败回归与修复后结果；实际测试 SHA/tree；introduced_by_sha；目标 remote/ref；push 回执；verified_on_sha及CI run/job；未验证/阻塞；当前状态。没有真实 commit/push/CI 回执时写 null 和原因，不填示例 SHA，不把 LOCAL_VERIFIED 写成 FIXED。

参考来源

以下官方文档支持 Git/CI 命令语义；本规程的单问题、隐私与放行标准是针对本次用户要求制定的工程规则。

- G01 git-push：refspec、porcelain 与推送选项
- G02 git-status：porcelain 与 NUL 输出
- G03 git-worktree
- G04 git-remote：get-url/push URL
- G05 git-diff
- G06 git-write-tree
- G07 git-commit
- G08 githooks
- G09 git-ls-remote
- G10 git-fetch
- G11 git-merge-base
- G12 git-cherry-pick
- G13 GitHub workflow syntax
- G14 GitHub workflow events 与实际 SHA
- G15 GitHub workflow 文件位置
- R01 参考 README
- R02 参考 pr.yml
- R03 参考 verify.py
- R04 参考 pyproject.toml

09 OpenBot macOS MVP：证据驱动的发布门禁

版本：2026-11-05。适用项目：xiongcp/OpenBot。本文件是验收规则，**不是已经通过验收的报告**。目前执行结果均从 NOT_RUN 开始，代码、测试、push、CI 和产物 SHA 只能在实际执行后填写。

1. 允许宣布的范围

默认目标为 **完整 macOS MVP**：按已经确认的 v1.2 / M3 功能基线，在明确的 Mac 硬件/架构、macOS build、安装路径和服务版本上，使全部原必需链路、安全边界、持久恢复、真实集成和交付行为通过验收。本轮限定的是平台证据范围，不删减原必需功能。它不能自动代表原工程计划 M3 的 Windows/Linux/macOS 三平台完成，也不能代表 M4 的 Tauri、签名公证桌面发布。

执行前冻结范围清单：当前交付的必需能力、明确不在本次发布中的能力、每个条件门禁的适用理由。原要求中的必需能力不能由执行代理自行删掉。能力已经声明提供，就必须验真；不能在看到失败后将其改为范围外。未知是否适用时先核对实际产品入口与原阶段，不填 N/A 掩盖缺口。**当前代码尚未实现的原必需能力应登记实现缺口，不能因此排除。**

真实云 Provider 与真实 Ollama 是两条独立必需链路，各自需要真实运行证据。只支持其中一种时，最多描述为收窄的 Mac 试点或局部验证，不能宣布完整 macOS MVP；完整目标仍有阻断。Tauri / M4 仍保持原阶段的条件门禁，不反向成为 CLI/daemon MVP 的无条件要求。

维度	可计入的证据	不足以支持的替代证据
Apple Silicon	真机 Darwin arm64；受测进程和所有 helper 的实际架构	测试 shell 是 arm64、手填 Darwin=true
Intel	声明支持时，在 Intel Mac 上原生运行	Apple Silicon 上的 Rosetta 或 Linux amd64 VM
Darwin 系统集成	用户登录会话中的 launchd、Keychain、文件系统、睡眠/网络等真实行为	Docker Desktop 内部 Linux VM 或修改对象时钟
CLI/daemon MVP	实际安装、跨进程本地 API、daemon/worker 启停、真实恢复与隔离	仅 in-process harness 或参考设计校验
Tauri/TCC/签名	若本次声称提供，对实际二进制和下载产物做真实验证	未签名试点、临时去 quarantine、关闭 Gatekeeper
三平台 M3	每个声明平台及架构自己的完整证据	macOS 通过后把 Windows/Linux 状态改成 PASS

关闭界面/客户端不应杀死长期驻留 daemon；用户明确停止或卸载才按身份和归属回收对应资源。无法提供隔离后端的危险动作返回不可用，不能转为宿主机执行。

2. 缺陷严重度与测试优先级是两条轴

缺陷严重度

等级	定义及示例	对当前 MVP 的处理
S0	未经授权产生外部效果、秘密泄漏、不可恢复数据损坏、隔离/身份边界失守	立即阻断发布；控制影响并修复，相关安全门禁全部重验
S1	必需功能不可用、可确定重复副作用、UNKNOWN/取消/恢复失真、安装无法完成、发布证据可被伪造	当前范围阻断发布；必须闭环
S2	局部非关键行为错误且有明确受控替代，未破坏安全与数据正确性	若触及任何必需需求仍阻断；只有已明确可延后的非必需项可保留 DEFERRED
S3	文字、布局或非关键体验瑕疵	可按原范围决定修复顺序；保留真实状态和影响

严重度不能只根据测试是否 P0/P1 推导；CI/测试入口错误也可能阻断全部验收。缺凭据本身是环境阻塞，不伪装成产品 bug 已修复；实际实现缺入口/适配器则属于实现缺口，不能一律归为缺环境。

测试优先级

优先级	内容	放行要求
P0	授权、持久化、恢复、取消、隔离、数据出口、真实能力与证据基础	所有适用必需用例 PASS；FAIL/BLOCKED/NOT_RUN 都不放行
P1	当前版本必需的真实端到端、安装与兼容性、已经承诺的用户行为	所有适用必需用例 PASS；不能用“优先级较低”跳过
P2	本次范围外或明确可延后的增强、探索性检查	保留范围与结果；不得把原必需项降为 P2 以求通过

S0/S1 未关闭数必须为 0；这一条件并不自动允许所有 S2/S3，也不能代替用例门禁。DEFERRED、BLOCKED 和 NOT_RUN 都不是 FIXED。

3. 证据等级和状态

每条用例同时记录测试层级、实际环境和产品路径，不能只有一个 passed: true。

层级	能证明什么	仍需补什么
合同/单元	Schema/JCS、状态守卫、明确输入输出、可控时钟和模型提案	实际 adapter、存储、进程、网络及服务行为
生产路径集成	真实 application/adapter/storage、SQLite、HTTP/stdio、故障注入	声明涉及的原生 OS 和真实外部服务
原生 macOS	Darwin、Keychain、launchd、进程组、文件系统、网络/睡眠等	未在本次机器/架构覆盖的变体
真实服务	专用飞书测试 app、真实云 Provider、真实本地模型、真实 Chromium/MCP	其他服务版本/账号配置、供应商不可见内部行为
干净安装/交付	固定候选构建产物在新用户/目标环境的首用、卸载和资产完整性	未测试架构/签名模式，不自动外推

现有 44 个 TC-CORE-* 与 38 个 TC-MAC-* / TC-INT-* 是执行清单，初始均 NOT_RUN。核心用例也必须调用产品实际模块与真实 SQLite；参考 DDL 的历史校验不是当前实现结果。原生和集成用例不能由布尔参数、手填 attestation 或模拟对象代替。[R1–R4]

单条执行状态使用：NOT_RUN、RUNNING、PASS、FAIL、BLOCKED_ENV、BLOCKED_CREDENTIALS、BLOCKED_DEPENDENCY、INCONCLUSIVE。测试脚本 skip 必须记录原因；只有执行前已证明不适用的条件用例才可用 NOT_APPLICABLE，且带原范围依据。mandatory case 的 skip、超时、collect 错误或部分运行不得记 PASS。

问题状态使用：OPEN、NOT_RUN、TRIAGED、REPRODUCED、FIXING、LOCAL_VERIFIED、COMMITTED、PUSH_UNKNOWN、PUSH_BLOCKED、PUSH_VERIFIED、CI_PENDING、CI_BLOCKED、FIXED、REOPENED、DEFERRED。状态允许保留多个维度：例如 push 已确认而 CI 仍阻塞；不能用一个“完成”字段抹去未完成门禁。

4. 必须逐项通过的门禁

表中的用例范围是入口索引，不限制需要追加的具体回归。执行者必须把每个 G 项映射到本次实际适用的用例 ID、命令、证据和候选 SHA；缺失映射本身阻断放行。

门禁	必需验收	典型关联用例/证据	阻断例
G00 仓库与固定候选	确认项目、push 目的地、分支、实际 baseline；最终 candidate SHA/tree 固定、可达；用户 dirty 内容未混入	repo binding、parent/tree/diff、远端精确 ref 回执	把只读参考 SHA 当本机基线；只测未提交目录；目的地不明
G01 可运行测试与 CI	当前真实命令可收集执行；必需依赖存在；workflow 位于支持的位置，事件/过滤命中；实际 job 运行	command manifest、collected/executed/skip、CI run/job/checkout SHA	collection errors、SDK skip冒充成功、子目录 YAML 未被发现
G02 真实内核链路	ingress→Task/Operation/Approval/Attempt→Broker→executor→真实效果→OutcomeVerifier→outbox；各入口与 DB 一致	TC-CORE-001–006、018、044	in-process transport 冒充 socket；直接 SQL 写成功；只断言能力标签集合
G03 授权与两次短事务	授权摘要不可变；过期/nonce/policy/actor/cancel/resource/epoch 在必要阶段复核；CONSUMED 不退回 APPROVED	TC-CORE-007、009–014、020–022；竞争屏障与事务轨迹	批准后提案被修改；只 claim 复验；旧 grant/epoch 继续派发
G04 UNKNOWN、取消与隔离资源	崩溃窗口保守 UNKNOWN；可信证据核对；取消终态无未决 DISPATCHING/UNKNOWN；新 ID 不绕过 quarantine	TC-CORE-015–017、019、023–024；独立效果接收器	因效果计数0就盲重跑；一句人工“成功”覆盖 DB；进程重启清 quarantine
G05 Writer 与存储故障	完整事务由唯一 writer 管理；稳定请求 ID 查 commit 结果；队列/背压；实际满盘/故障不触发未记账效果	TC-CORE-025–029；真实 SQLite/文件系统故障	caller 超时当 rollback；硬编码 blind_retry=0；未执行就追加 disk_full 标签
G06 Artifact、补丁、备份恢复	旧/新 hash 与审批精确绑定；多文件部分提交可恢复；备份 DB/Artifact 一致并 pin GC；secret_ref 与真实秘密分开	TC-CORE-030–033、TC-MAC-018、021	parser 成功当安全写入；备份缺 artifact仍成功；真实秘密入备份
G07 身份、策略、秘密与出口	本地认证、owner/对象作用域、硬拒绝、注入边界、用途最小凭据、日志脱敏、实际 egress	TC-CORE-008、034–038；TC-MAC-019、022；TC-INT-014	外部 hints/allowlist 提权；scanner联网；localhost 当无云证据
G08 Provider 合同与真实双类型模型	SDK 真实请求计数/唯一重试负责人；流式工具参数完整；禁自动执行 helper/托管工具旁路；预算/取消；至少一个真实云 Provider和真实 Ollama 本地模型分别留证	TC-CORE-039–043；TC-INT-013、016及明确云模型子项	mock Ollama 代表云和本地；取消后新 HTTP 请求；手填计费/egress attestation

门禁	必需验收	典型关联用例/证据	阻断例
G09 原生 macOS 运行与隔离	实际用户服务、唯一 profile 锁、正确进程身份、客户端退出不杀 daemon；真实睡眠/网络/重启；Docker生命周期和回收次序	TC-MAC-001、003、008–019	父子重复排他锁自锁；只杀 CLI不收容器；缺沙箱转宿主机
G10 真实飞书链路	专用测试 app、单 gateway、owner身份与撤销、durable ACK去重、卡回调时钟/重投、发送 UNKNOWN、进程外 watchdog	TC-INT-001–006、016	只用直接调用回调对象；发送副作用重复；阻塞 SDK让控制面失联
G11 受控 Browser 与 MCP	真实 Chromium sandbox/资产/profile锁；Service Worker/重定向出口；下载上传；真实 stdio、manifest变更、超时核对	TC-INT-007–012、015	route 拦截当全部出口控制；no-sandbox静默；tool hints绕过审批
G12 安装、离线资产、恢复与供应链	干净Mac用户真实首用；锁定依赖和全部运行资产来源/hash/许可证；声明交付产物非空且逐个验证；恢复secret_ref失效可处理	TC-MAC-004–005、010、021；实际产物manifest、SBOM/来源/锁和构建记录	开发机缓存掩盖下载；只数manifest文件；verified_artifacts=0仍 pilot_ready
G13 累计回归和长运行证据	当前候选重跑必需累计回归；真实故障/恢复、资源与预算观测；样本数、时长、峰值有原始依据	实际 soak/chaos日志、故障事件与计数、前后资源、前次修复回归	修改对象时间冒充睡眠；零样本通过；承诺未测性能百分位
G14 对准候选的 CI 与最终结论	本地、真实门禁、CI、构建对应同一候选；CI实际checkout核对；每个阻断理由直接否决 ready	immutable candidate、run/attempt/job、证据索引与release评估负例	其他 SHA 绿灯嫁接；布尔ready输入；有blocking_reason仍ready
G15 已声明的条件交付能力	Intel、Tauri/TCC、签名公证/离线Gatekeeper、升级仅按事先声明范围启用门禁	TC-MAC-002、006–007、020–021的适用部分	Rosetta冒充Intel；去 quarantine后说首次启动通过；失败后改为范围外

外部效果必须由独立观察方证明

效果接收器的账本与被杀的 daemon、它的 DB 和进程分离；用实际文件 hash、服务端持久 receipt 或有稳定 operation key 的受控接收器核对。记录至少三个不同指标：客户端/socket发送、服务端接收、逻辑效果。安全幂等重传可能带来多次请求但只产生一个效果，不能把所有计数强行写成 1。

DISPATCHING 已落盘后、发送前被 kill，即使测试知道服务器效果为 0，产品仍需要 UNKNOWN 和受信核对；测试信息不能越权成为生产恢复依据。所有关键竞态都要用屏障/事件分别控制两种线性化顺序，保留真实事务/进程时刻，而不是只重复 sleep 测试。

真实服务只在专用目标上产生受控效果

真实飞书用已授权测试 app/目标，真实模型使用最小必要请求和已知预算，浏览器/MCP写测试使用隔离的受控对象。凭据缺失可阻塞 G08/G10 的对应子项；不影响继续 G02–G07 等独立工作，但仍阻断最终范围完整放行。不得把真实用户数据或任意公开站点当故障注入目标。

长运行不以伪精确承诺代替测量

执行前记录计划持续时长、并发、工作负载、资源预算与停止条件。原需求已有具体阈值时遵守；未定义时先制定可审查的试验方案，报告实际样本和区间，不能自行声称“24小时无故障”“P99低于某值”或零泄漏。没有真实故障注入就不能报告相应恢复通过；进程/容器/连接/队列增长必须来自实际观测。

5. 固定候选 SHA 与 CI 门禁

修复引入提交叫 introduced_by_sha；实际通过所有对应验证的提交叫 verified_on_sha。通常相同。若后续修复另一个独立 CI 基线问题，则后者可以是包含前者的后续候选，但必须验证 ancestry、重跑相关测试，不能直接搬用旧结论。

最后冻结 candidate_sha / candidate_tree，所有最终必需测试和 CI 以它为对象。构建产物注明 source_sha、锁/依赖/资产摘要、构建步骤和实际文件 hash。提交自身不需要把自身 SHA 写进源文件；回执和发布清单在 Git 外生成并绑定候选，避免无限 amend/重建自引用。

PR CI 可能 checkout 合并合成提交，run 元数据与实际 checkout 也可能不同。按真正测试的代码记录三者，不能把合并提交结果当修复 head 结果。要验证修复 head，选择实际支持的 push / dispatch 工作流或明确 checkout它；要发布 merge候选，则把它作为新候选重新核对。只检查“有一个成功的 CI run”不合格。[G1][G2]

GitHub 仅识别支持位置的 workflow；在 .github/workflows/ 子目录放 YAML 不满足自动发现要求。[G3] 当前参考 run 41949551726 已失败，且上述 runner / Python / collection / 依赖安装存在独立问题线索。任何修复后的结论必须由新的真实 run 支持，不能把这些历史记录覆盖成 PASS。

可以在某个修复已经 push 并被远端确认后，保留 CI_BLOCKED 并继续修另一个独立 CI 根因；这只解决执行顺序死锁。**CI_BLOCKED 仍然不能放行，也不能把问题记为 FIXED。** push 未确认则不能累积下一项修复提交。

6. 发布评估器不能信任自报成功

发布结果必须从不可变执行记录导出，并验证来源、范围、代码和覆盖率。不能仅收集调用者传入的 security_critical_failures=0 / darwin_arm64_verified=true。参考 mvp_release_gate.py 静态审查提示，存在已记录供应链阻断理由而 pilot_ok 仍可为真的路径；本机需要用受控负例复现，并按单问题流程修复。[R4]

最小必要判断语义如下，属于验收规则，不是仓库中已经存在的 API：

```
先验条件：冻结且非空的 required_case_ids、required_gate_ids、expected_artifact_ids
候选条件：真实 candidate SHA/tree + 精确远端可达证据
证据条件：每条必需记录来源可核、未被篡改、关联当前 candidate 和环境
执行条件：每条必需用例实际执行、断言成立、无未解释 skip/超时/collect 错误
产物条件：声明交付清单中的每个产物存在并通过 hash/来源/完整性核验
CI条件：必需工作流/作业真正运行，实际 checkout 对准当前候选，结果通过
缺陷条件：S0/S1为0；不存在任何触及必需需求的未关闭缺陷
阻断条件：blocking_reasons 必须为空；任何一条均否决 ready
只有以上全部成立才可生成 PASS_MACOS_MVP
```

必须添加负例证明：空用例集、零执行数、零验证产物、缺证据、伪造 PASS、其他 SHA 的绿灯、未知 OS/arch、未触发 workflow、只有 mock 的真实服务项、任一 blocking_reason，都不会返回 ready。测试数量为 0 不能因“没有失败”而通过。

参考实现的 capability 名称集合、硬编码危险重试计数、未执行的 disk_full 标签和固定成功 Ollama transport，只能证明代码走过对应分支，不能满足 G05/G08/G09/G13 的现实证据。[R1–R3]

7. 证据保存与可公开摘要

原始证据放仓库外受限目录，并限定保留时间与采集范围。原始 HTTP、header/cookie、凭据、个人路径、聊天内容、用户 DB、截图、飞书账号/租户标识不得进入 Git 或公开 CI artifact。需要外发的摘要须去除秘密和 PII，只保留合成 fixture、相对源码路径、版本、用例 ID、退出码、数量、期望/实际以及不透明证据引用。

私有运行账本记录原始文件位置与完整性校验；公开摘要不暴露原始用户路径，也不以秘密哈希替代秘密。扫描离线运行，外部 allowlist 注释不生效，并做人工内容审查。记录命令 argv 中的安全参数，不记录敏感环境变量值。凭据是否可用可以记录，值不可记录。

8. 最终输出与放行结论

最后必须交付一个具体候选的：范围表、命令清单、用例结果、缺陷台账、每个修复的 commit/push/CI 回执、构建/产物清单、脱敏证据索引和尚未满足项。不得只有“全部测试通过”的一句话。

结论	含义
NOT_RUN	尚未执行本次完整验收；本方案当前初始状态
FAIL	已执行的必需行为不符合要求
BLOCKED	环境、凭据、CI、证据或其他依赖使必需门禁无法完成；可以有局部 PASS
PASS_MACOS_MVP	固定候选在列明的 macOS/架构/服务/交付范围内全部必需门禁通过

FAIL 和 BLOCKED 可同时有多个原因，报告不要只保留其中一个。Windows/Linux 或未声明 Intel/M4 桌面能力保持各自真实状态；不能在最终表中填成通过。新提交、新产物、新运行配置或能力声明改变了测试对象，就要按影响重新验证，旧候选结论不得自动继承。逐问题 commit+push 的授权不自动包括 merge、tag、正式部署。

来源与核对入口

以下源文件是只读参考 SHA 的实现线索，执行时须核对本机当前版本；引用并不表示该实现已经通过测试。

- R1 chaos_soak.py
- R2 mvp_regression.py
- R3 macos_gate.py
- R4 mvp_release_gate.py
- R5 参考 CI run 41949551726
- G1 GitHub workflow syntax
- G2 GitHub events 与 SHA
- G3 GitHub workflow 文件位置

10 可直接交给本机 Codex / Claude Code 的执行提示词

> 使用方式：在用户 Mac 的 OpenBot 实际工作副本中，将以下正文作为执行任务。配合docs/07_fix_commit_workflow.md、docs/08_release_gate.md、实际用例矩阵和 templates/workflow/ 使用。此提示词没有声称任何测试、修复、提交或推送已执行。

你要验收并修复已经存在的 OpenBot 实现。不要重建项目、用新脚手架替代当前代码，或仅输出建议。工作目标是在实际 Mac 环境验证严格 MVP，发现真实问题后逐个完成最小修复和回归，并且**每修复一个独立问题就 commit、push 一次**。

一、已知项目与授权

项目已经确认是 https://github.com/xiongcp/OpenBot。远端只读审查参考过 main 的 a223d36a659b7ecf32fca2b4890fc187d3e96e48，但你必须从当前本机 checkout 重新记录实际基线，不能把这个参考 SHA 当作当前 HEAD 或已经验证的候选。

用户已授权本轮逐问题 commit 和 push。仓库 AGENTS.md 中“未经授权不推送”的前提已经满足，**不要每完成一个修复再问用户是否允许提交/推送**。继续遵守其他项目规则，不修改 AGENTS.md 制造永久授权。此次授权不等于自动 merge、改保护规则、向 main 发布、打 release tag 或部署。

先检查当前 cwd、真实仓库、分支、上游、HEAD/tree、dirty 状态、已有暂存内容、未推送提交，以及真实 remote 的 fetch URL 和 **push URL**。优先核对 origin，但名称本身不是目的地证明。默认使用明确的现有开发分支，或从已确认基线建立 mvp/macos-hardening 修复分支；不能自动向 main 写入。仅在本机目录无法定位、多个工作副本/目的地互相冲突、push URL 指向其他仓库、已有分支归属不明等具体问题无法解析时，向用户询问最少缺失信息。已知项目地址不要再问一次。

原工作区有用户改动时，保留其文件、index、未跟踪内容和已有提交；优先新建自己拥有的独立 worktree。不要自动 stash、reset、clean 或搬运用户未提交内容。不能把用户既存未推送提交夹带到本轮发布范围。任何自己的临时工作树和日志目录，都先明确归属。

二、先建立可复核的执行基线

在当前代码读取实际 AGENTS.md、README、pyproject.toml、锁文件、scripts、CLI parser、测试与 CI 配置，产出命令清单。每条记录 argv、cwd、代码来源、环境键名、依赖组、超时、是否需要真实服务、是否有外部副作用。不要用 eval 执行文档文本，也不要臆造 OpenBot 命令。

参考版本存在以下真实入口，先在当前 checkout 复核：

```
uv sync --locked
.venv/bin/python scripts/verify.py
uv run --no-sync pytest --collect-only -q
uv run --no-sync pytest
```

README 使用前两条，当前参考 CI 使用最后一条 pytest console 入口。python -m pytest 的绿灯不能掩盖 uv run --no-sync pytest 的 collection/import 失败。

参考 scripts/verify.py 会设置 core 与 feishu_gateway 的 PYTHONPATH、移除 OPENBOT_NATIVE_TESTS、默认 phase M0 且仅接受 M0/M1，并为每条命令设置 60 秒超时。它把原始日志写到 evidence/implementation。不要因此宣称原生 Mac、真实服务或 M3 已通过；不要把这些原始日志自动加入 Git。默认依赖组为 dev-contracts，额外组有 provider-openai；需要时按当前已锁定组安装，例如参考版本的 uv sync --locked --group provider-openai。缺依赖导致 skip 不等于测试通过。

以下是参考 SHA 的只读问题线索，需在本机当前版本复核，不能直接标记为本轮已复现或已修复：

- pr.yml 的真实命令为 uv run --no-sync pytest；参考 run 41949551726 的 Linux job 有 24 个 No module named tests collection errors，Mac/Windows 在 setup-python 阶段因所选 Python 3.12.12/架构构建不可用而失败，另有 SDK skip。不要用“环境问题”四个字忽略可修复的导入和配置根因。
- 参考 matrix 使用 ubuntu-24.04 / macos-14 / windows-2022。重新核对受支持的 runner image、Python 构建与实际架构；不要沿用不可运行的组合。
- platform-macos.yml 只在 PR / workflow_dispatch 触发，参考配置没有依赖安装；push 到修复分支不自动证明它运行过。
- 部分 soak/gate/supply-chain workflow 位于 .github/workflows/ 子目录；GitHub 不支持在其子目录发现 workflow。YAML 文件存在不代表 CI 已执行。

这些是多个可能独立根因，要分别登记、验证、修复、提交和推送。恢复测试入口与 CI 可执行性优先于宣称产品已通过完整验收。

三、严格限定验收范围与证据

默认本轮交付声明是 **完整 macOS MVP**：按已经确认的 v1.2 / M3 功能基线在 macOS 上验收，保留全部原必需功能与安全边界，只限定本轮平台证据范围。真实云 Provider 与真实 Ollama 是两条独立必需链路，不能二选一。只支持一种模型服务时，只能如实描述为收窄的 Mac 试点或局部通过，完整 macOS MVP 仍未完成；不得执行代理自行降低要求。当前代码未实现的原必需能力是实现缺口，不能作为排除理由。实际 macOS 版本、芯片架构、原生/Rosetta、安装方式、模型和服务版本必须写明。macOS MVP 通过不能改写为原 M3 三平台完成。Windows / Linux 没有真实证据就保持 NOT_RUN。Intel 原生、Apple Silicon 原生和 Rosetta 分别记录；Docker Desktop 的 Linux VM 不证明 Darwin、TCC、launchd 或钥匙串行为。

Tauri 界面、TCC/GUI、签名、公证、自动升级等，按当前版本实际声明和原阶段范围判断。若只交付不含这些功能的 CLI/daemon MVP，可明确为范围外；不能因环境缺失或测试失败临时改为范围外。若交付声称已包含这些能力，就必须执行相应真实门禁。

采用 docs/08_release_gate.md 的 S0—S3 缺陷严重度和 P0—P2 测试优先级，二者不是同一字段。所有当前范围必需 P0/P1 用例需要有真实 PASS 证据；S0/S1 不能留存放行。S2/S3 也不能绕过必需需求或安全门禁。

现有测试和参考设计都要复用，但测试必须穿过实际生产 application/adapter/storage 路径。参考 DDL 或 mock 的绿灯不能替代生产路径、原生 Mac、真实 Provider、真实飞书、真实隔离或安装证据。所有尚未执行的用例从 NOT_RUN 开始，不填示例成功次数、假 SHA、假 CI URL 或不存在的命令。

特别检查：

1. CLAIMED / DISPATCHING 的持久化、取消、epoch/resource 复核，审批不可篡改、CONSUMED 不退回 APPROVED，Attempt dispatch_id 与 Operation provider idempotency key 的边界。
2. 独立接收器记录真实外部效果；分别计算 socket send、server received 和 logical effect。幂等请求可重传，但逻辑效果不能重复。不能一句“exactly once”省略证据。
3. DISPATCHING 已落盘但发包前被杀，即使夹具观测到效果 0，生产状态也应保持 UNKNOWN，直到可信回执/reconcile 证明如何处置。不能用测试全知视角直接盲重试。换 Task/Operation ID 不能绕过持久 quarantine。
4. 取消终态必须满足其辖下不存在未解决 DISPATCHING / UNKNOWN；竞态用屏障/事件固定前后顺序，覆盖两种线性化结果，不靠 sleep 随机碰撞。
5. 宿主机执行不能作为缺沙箱时的隐式后备。Docker CLI 退出不等于容器消失，回收需 stop/wait → 导出及校验 → remove。daemon 和 worker 的生存期不能绑桌面 UI。
6. localhost 的模型服务不证明本地离线；用实际选定模型、禁云配置和真实 egress 观测证明。Provider SDK 内建重试和工具执行 helper 不得绕过控制面。
7. 原始证据的来源要核对。现有 chaos_soak 标签、硬编码 0、固定成功 mock、手填 egress attestation、布尔型 release evaluate 输入，不自动算真实满盘/断网/睡眠/云模型/供应链/发布成功。发现逻辑缺口先受控复现，再按单问题闭环修复。

四、每个问题完整执行一次闭环

只指定一个集成代理操作 index、commit 和 push；其他代理可以并行做独立审查、隔离复现和只读验证，不同时争用同一工作树。

每次选一个独立根因，执行：

1. 建立 issue_id、S级、关联用例/P级、base SHA、期望/实际、最小复现和证据。先运行基线，区分产品错误、入口/CI 错误、真实环境缺口和测试本身错误。

2. 新增或扩展有意义的失败回归；已有稳定用例可直接纳入，不复制无意义测试。证明修复前失败点与根因一致。配置问题使用真实配置识别/实际 job 证据，不写仅断言某字符串存在的镜像测试。
3. 做最小修复，运行目标回归、具体受影响回归以及项目必需检查。安全/状态修改必须含对应 P0 反例。保留以前修复的相关回归，禁止以删除断言、加 skip、扩权、无条件成功或伪造结果过关。
4. 若完整基线已有另一独立失败，明确保存其证据和关联 issue；本问题的针对验证与可运行相关检查必须通过。不得把自己新引入的失败称为基线问题。CI 才能验证的配置结论等到实际 CI 后才能闭环。
5. 明确审查并仅暂存该问题具体路径，不用 git add -A、整目录或不受控 glob。检查暂存内容没有用户改动、秘密、PII、真实 DB/profile、原始日志或无关生成物。
6. 按项目约定创建一个原子修复 commit，使用现有真实作者和签名配置。不要为工具额外加 attribution。记录实际 parent、tree 和文件范围；hooks 修改内容时重新审查/测试，不使用 --no-verify 绕过。
7. 在新建干净 checkout 上针对实际 commit SHA 运行相关验证。先前工作树有未跟踪文件或 hook 改动参与测试时，不能把旧结果挂到新 commit。
8. 使用已确认 remote 和完整 refs/heads/...，把精确修复 SHA 推到明确目标。禁 force、mirror、all、附带 tag 或悄悄发布其他子模块。
9. push 无论返回成功还是超时，都实时核对实际 push 端点目标 ref。相同 SHA 已到达就记已确认，不重复 commit。远端已前进则证明 ancestry；只有可达不等于新的 tip 已通过测试。查不清就 PUSH_UNKNOWN，不能猜。
10. 核对实际被识别、被触发的 CI workflow、run/attempt/job、依赖组、runner OS/arch 和真正 checkout SHA。PR 合成 merge SHA 与修复 head 不能混用。相关真实门禁与 CI 对齐 verified_on_sha 后才能 FIXED。

提交自引用不需要解决：修复提交可以包含 issue_id、base_sha、run_id 和脱敏结果，**不要要求它包含自身 SHA**。commit/push/CI 回执写到 Git 外私有运行账本；最终可另作汇总报告，不为每个问题再 amend 或补一个纯记账 commit。

五、阻塞和后续回归的规则

- **最多一个尚未确认推送的修复候选。** push 失败或结果未知，先解决它；可继续盘点、独立复现或补证据，不形成第二个代码修复/未推送提交。
- push 超时先查远端；已成功不重复提交。确实未到达可对同一 SHA 有界重试。不能因为超时擅自 force。
- non-fast-forward 先看最新远端。仅对确定未发布的本问题候选，可从已确认新基线重建并重新测试；旧候选记 SUPERSEDED_LOCAL。已发布或发布未知的提交禁止改写。
- 认证/保护分支阻塞时使用用户已有合法认证；必要登录在用户本机完成，不索要明文 token、不放宽权限、不关闭分支保护。若需要改变用户明确指定的目标，再解释具体冲突并确认该改变。
- 上一项 **已确认 push**，但 CI 被另一项已识别的独立基线故障挡住，可以保留 CI_BLOCKED，继续单独修那个 CI 根因并 commit/push；不能提前 FIXED。最终用后续候选重新跑相关测试。记录 introduced_by_sha 与 verified_on_sha，证明后者包含前者。
- 缺真实飞书/模型凭据或原生环境时，只阻塞对应门禁，继续独立可完成工作。不要伪造成功，不把缺环境变成 N/A，不让 release gate 忽略它。
- 后续修改使已通过用例回归，要使旧结论 REOPENED/失效；用新修复 commit 或明确 revert 修复并独立 push。已推送历史不 amend、rebase 或 squash。
- 同一失败偶尔重跑变绿不等于根因消失，保留失败样本与时序证据。发布候选必须运行最终累计的相关回归。

六、证据与汇报

原始证据放在仓库之外的私有、权限受限目录。真实 HTTP/headers、cookie、token、飞书用户/租户 ID、用户路径/文件内容、DB 和截图先最少采集，再保存；不要进入模型上下文、公开日志、Git 或 CI artifact。可入 Git 的只有审核过的合成夹具、相对路径、用例/版本/计数、脱敏摘要和不透明证据 ID。秘密扫描离线执行，外部 allowlist 注释不获得豁免；扫描阴性也需要内容审查。

使用给定 JSON 模板并在实际执行后填写。每条命令保留退出码、时间、代码 SHA/tree、环境、收集/执行/失败/skip 数及具体 skip 原因。introduced_by_sha、verified_on_sha、remote ref、CI URL、artifact hash 都只能填实际观测值，没有就 null。

持续简短汇报正在验证的风险、已确定根因、当前问题状态和阻塞。每项结束报告 issue_id、根因、文件、回归、commit SHA、push 目标与远端回执、同代码 CI/真实门禁结果。没有做过的步骤直接写未做。

最后冻结一个实际候选 SHA，完成 docs/08_release_gate.md，提交：

- 当前 Mac/架构/安装/服务范围、真实命令清单和最终候选 SHA/tree；
- 全部用例结果、原始证据的安全索引和脱敏摘要；
- 每个问题的 introduced_by_sha、verified_on_sha、push/CI 回执和后续回归关系；
- 明确剩余 NOT_RUN / BLOCKED / FAIL 与所缺条件，不把它们折成 PASS；
- 结论仅为证据实际支持的 macOS MVP 范围，不把本次结论扩张成三平台或正式签名桌面发布。

如果目标尚未完成，继续所有已授权且独立可做的工作；只有客观缺失信息、凭据或权限挡住对应动作时说明具体原因。不要以“已提供计划”替代本机实际执行。

11 100项用例索引与执行登记

本目录完整操作卡见 cases/TEST_CASES.md，机器可读清单见 cases/test_cases.json。全部尚未执行，NOT_RUN不是失败，也不是通过。只对事先冻结范围外的条件用例使用NOT_APPLICABLE。

基线与验收真实性

ID	用例	优先级	真实环境	初始状态
TC-QA-001	源码、dirty状态与远端归属冻结	P0	需要	NOT_RUN
TC-QA-002	锁定Python与依赖在实际架构可安装	P0	需要	NOT_RUN
TC-QA-003	本地与CI测试收集一致	P0	需要	NOT_RUN
TC-QA-004	验证超时和异常不得沿用旧PASS	P0	模块集成	NOT_RUN
TC-QA-005	真实console入口与安装路径	P0	需要	NOT_RUN
TC-QA-006	工作流被识别并执行真实门禁	P0	需要	NOT_RUN
TC-QA-007	模拟证据不能升级为原生资格	P0	模块集成	NOT_RUN
TC-QA-008	发布证据绑定当前候选与范围	P0	模块集成	NOT_RUN

内核、授权、恢复与Provider

ID	用例	优先级	真实环境	初始状态
TC-CORE-001	真实本地入口贯通四对象与一次受控效果	P0	需要	NOT_RUN
TC-CORE-002	Task 合法跃迁、终态守卫与只读回放	P0	模块集成	NOT_RUN
TC-CORE-003	入站去重、负载冲突与身份作用域	P0	模块集成	NOT_RUN
TC-CORE-004	T1 提交前与提交后 ACK 丢失的恢复	P0	需要	NOT_RUN
TC-CORE-005	状态、审计、outbox 与完成结果的事务原子性	P0	模块集成	NOT_RUN
TC-CORE-006	严格 JSON、JCS 标准向量与本地 Schema 边界	P0	模块集成	NOT_RUN
TC-CORE-007	已批准提案不可变及旧新内容精确绑定	P0	模块集成	NOT_RUN
TC-CORE-008	真实本地 API 身份、作用域与对象归属	P0	需要	NOT_RUN
TC-CORE-009	审批 nonce、TTL 与重启后的 challenge 保护	P0	需要	NOT_RUN
TC-CORE-010	同版本审批竞争与批准后合法撤销	P0	模块集成	NOT_RUN
TC-CORE-011	批准已提交但响应丢失不产生第二授权	P0	需要	NOT_RUN
TC-CORE-012	撤销 owner 或授权与 claim 的确定性竞态	P0	模块集成	NOT_RUN
TC-CORE-013	每次派发前复验期限、策略、身份与资源	P0	模块集成	NOT_RUN
TC-CORE-014	CLAIMED 后取消与第二派发事务的线性化	P0	模块集成	NOT_RUN
TC-CORE-015	claim 已提交、尚未派发时的真实进程崩溃	P0	需要	NOT_RUN
TC-CORE-016	DISPATCHING 已提交、发包前崩溃仍须核对	P0	需要	NOT_RUN
TC-CORE-017	外部效果成功、回执落盘前崩溃不盲重放	P0	需要	NOT_RUN
TC-CORE-018	结果已提交、通知或 UI 响应丢失不重执行	P0	需要	NOT_RUN
TC-CORE-019	在途取消保留未知结果且阻止后续动作	P0	需要	NOT_RUN
TC-CORE-020	同一 Attempt 重传与 dispatch 接收去重	P0	需要	NOT_RUN
TC-CORE-021	同 Operation 安全重试、稳定幂等键与失效拒绝	P0	模块集成	NOT_RUN
TC-CORE-022	旧 epoch、过期租约与迟到结果不能制造双执行者	P0	需要	NOT_RUN
TC-CORE-023	UNKNOWN 资源 quarantine 防新 Task/Operation ID 绕过	P0	模块集成	NOT_RUN
TC-CORE-024	核对证据、Operation.version 与人工确认来源	P0	模块集成	NOT_RUN
TC-CORE-025	单 writer 独占完整事务与连接约束	P0	模块集成	NOT_RUN
TC-CORE-026	调用者取消或超时后查询真实提交结果	P0	需要	NOT_RUN
TC-CORE-027	队列预留、公平性、总 deadline 与背压	P0	模块集成	NOT_RUN
TC-CORE-028	派发前真实 SQLite 满盘或提交失败	P0	需要	NOT_RUN
TC-CORE-029	外部成功后满盘导致结果不可持久化	P0	需要	NOT_RUN
TC-CORE-030	在线备份与并发 GC 保持 DB/Artifact 一致	P0	需要	NOT_RUN

ID	用例	优先级	真实环境	初始状态
TC-CORE-031	缺失或被篡改备份的恢复必须拒绝假完整	P0	需要	NOT_RUN
TC-CORE-032	多文件补丁部分提交与用户脏改动保护	P0	需要	NOT_RUN
TC-CORE-033	Artifact 文件与数据库登记之间的崩溃缝隙	P0	需要	NOT_RUN
TC-CORE-034	硬拒绝与本地确认不能被批准或工具 metadata 覆盖	P0	模块集成	NOT_RUN
TC-CORE-035	提示注入、摘要与导入元数据不能洗白权限或数据	P0	模块集成	NOT_RUN
TC-CORE-036	OFFLINE、LOCAL_MODEL 与实际网络出口	P0	需要	NOT_RUN
TC-CORE-037	秘密按用途最小注入与凭据不可用时拒绝	P0	需要	NOT_RUN
TC-CORE-038	跨分片秘密过滤、异常日志与 scanner 隐式外发	P0	模块集成	NOT_RUN
TC-CORE-039	实际 Provider 请求计数与唯一重试负责人	P0	需要	NOT_RUN
TC-CORE-040	流式工具参数完整性、ID 对应与异常输出	P0	模块集成	NOT_RUN
TC-CORE-041	SDK 自动工具 helper、托管工具和扩展字段旁路	P0	需要	NOT_RUN
TC-CORE-042	Provider 取消、断流、deadline 与未知计费	P0	需要	NOT_RUN
TC-CORE-043	并发预算预留、重复结算与空转熔断	P0	模块集成	NOT_RUN
TC-CORE-044	outbox 重复、乱序与未知 ACK 不重执行业务	P0	需要	NOT_RUN

macOS原生环境

ID	用例	优先级	真实环境	初始状态
TC-MAC-001	Apple Silicon 原生身份与通过报告必须来自真实能力	P0	需要	NOT_RUN
TC-MAC-002	Intel 原生资格与 Rosetta 结果分开	P1	需要	NOT_RUN
TC-MAC-003	Darwin 控制面与 Linux 容器执行域必须分离声明	P0	需要	NOT_RUN
TC-MAC-004	干净 macOS 用户的真实安装与首次产品启动	P0	需要	NOT_RUN
TC-MAC-005	OFFLINE 干净首用不隐式补下载资产	P0	需要	NOT_RUN
TC-MAC-006	Gatekeeper 真实首开、签名与公证声明一致	P1	需要	NOT_RUN
TC-MAC-007	离线 Gatekeeper 与制品篡改负例	P1	需要	NOT_RUN
TC-MAC-008	launchd 登录会话中的真实服务注册和环境	P0	需要	NOT_RUN
TC-MAC-009	唯一 profile 锁、父子不自锁与客户端退出	P0	需要	NOT_RUN
TC-MAC-010	显式停止与卸载仅回收已登记资源	P0	需要	NOT_RUN
TC-MAC-011	真实睡眠/唤醒后的审批、连接与在途任务	P0	需要	NOT_RUN
TC-MAC-012	真实断网、网络切换与半开连接后的恢复	P0	需要	NOT_RUN
TC-MAC-013	注销/登录与受控重启恢复，不承诺登录前常驻	P0	需要	NOT_RUN
TC-MAC-014	真实 SIGKILL、多代/脱离进程与 PID 身份核验	P0	需要	NOT_RUN

ID	用例	优先级	真实环境	初始状态
TC-MAC-015	Docker 客户端死亡、引擎重启与容器独立归属	P0	需要	NOT_RUN
TC-MAC-016	实际容器权限、出口与资源上限生效	P0	需要	NOT_RUN
TC-MAC-017	容器产物 stop/wait→校验→登记→remove	P0	需要	NOT_RUN
TC-MAC-018	APFS 路径、大小写/Unicode别名与真实权限	P0	需要	NOT_RUN
TC-MAC-019	真实 Keychain 后端、锁定/拒绝与最小注入	P0	需要	NOT_RUN
TC-MAC-020	TCC 按实际功能和执行二进制最小授权	P0	需要	NOT_RUN
TC-MAC-021	macOS 数据备份/恢复与版本变更的真实重启	P0	需要	NOT_RUN
TC-MAC-022	真实SDK/系统故障诊断不泄漏秘密或启用遥测	P0	需要	NOT_RUN

真实服务与产品集成

ID	用例	优先级	真实环境	初始状态
TC-INT-001	飞书真实测试应用接入、配对与 owner 撤销	P0	需要	NOT_RUN
TC-INT-002	真实飞书事件持久 ACK、重投去重与重启	P0	需要	NOT_RUN
TC-INT-003	真实审批卡 callback 的 deadline、重复与失效	P0	需要	NOT_RUN
TC-INT-004	普通卡、CardKit权限不足与三个独立时钟	P0	需要	NOT_RUN
TC-INT-005	飞书发送成功但本地回执丢失的 UNKNOWN 核对	P0	需要	NOT_RUN
TC-INT-006	真实 SDK 阻塞与进程外 Gateway watchdog	P0	需要	NOT_RUN
TC-INT-007	真实 Chromium 启动、跨进程 profile 锁与沙箱	P0	需要	NOT_RUN
TC-INT-008	浏览器 Service Worker、重定向与实际出口	P0	需要	NOT_RUN
TC-INT-009	浏览器目标变更与点击成功后断线 UNKNOWN	P0	需要	NOT_RUN
TC-INT-010	MCP 真实 stdio/SDK 握手、环境与流量上限	P0	需要	NOT_RUN
TC-INT-011	MCP 工具动态变化、伪造 hints 与旧授权失效	P0	需要	NOT_RUN
TC-INT-012	MCP 写动作超时、取消与副作用核对	P0	需要	NOT_RUN
TC-INT-013	Ollama 本地模型来源、禁云配置与实际出口	P0	需要	NOT_RUN
TC-INT-014	OFFLINE/LOCAL_MODEL 实际受管域出口和模式切换	P0	需要	NOT_RUN
TC-INT-015	浏览器上传/下载和污染数据进入真实 Artifact 流	P0	需要	NOT_RUN
TC-INT-016	真实 macOS 产品从飞书任务到批准执行、重启和核对的闭环	P0	需要	NOT_RUN

逐问题提交推送

ID	用例	优先级	真实环境	初始状态
TC-GIT-001	单问题提交和用户改动隔离	P0	需要	NOT_RUN
TC-GIT-002	push结果不明的幂等核对	P0	需要	NOT_RUN
TC-GIT-003	非快进与并发远端更新	P0	需要	NOT_RUN
TC-GIT-004	CI状态必须对应验证SHA	P0	需要	NOT_RUN
TC-GIT-005	已推送修复引入回归时追加独立修复	P0	需要	NOT_RUN
TC-GIT-006	Git摘要与原始证据分离	P0	需要	NOT_RUN

任务质量与长期运行

ID	用例	优先级	真实环境	初始状态
TC-EVAL-001	冻结的24个真实任务场景	P0	需要	NOT_RUN
TC-EVAL-002	预算、请求数、成本与延迟可核对	P0	需要	NOT_RUN
TC-EVAL-003	真实时长浸泡与资源稳态	P1	需要	NOT_RUN
TC-EVAL-004	文档示例和能力矩阵与产品一致	P1	需要	NOT_RUN

12 来源、包内容与验证边界

本次实际完成的工作

读取了xiongcp/OpenBot固定SHA的选定生产代码、测试入口、任务状态与CI配置，并读取同SHA的contracts CI运行41949551726及三个失败job日志。形成16项初始待分诊线索、100项待执行用例、七份证据模板和逐问题修复交接规程。没有在用户Mac运行产品，也没有修改、提交、推送任何OpenBot仓库代码。

本执行包保留v1.2工程设计及现有实现作为输入；它不替代实际仓库中的AGENTS.md、锁文件和当前代码。源码观察不是漏洞实测；相应文件之外可能存在其他已实现路径，必须通过真实入口追踪和复现后再定根因。任务状态可能更新滞后，不能凭IN_PROGRESS就认定代码不存在。

使用包内文件

路径	用途
docs/00–04	范围、真实命令、16项线索、E0–E7执行顺序、质量与诊断
docs/05–06	核心故障注入、macOS与真实集成详细步骤及官方依据
docs/07–09	逐问题commit/push、G00–G15发布门禁、完整Codex执行提示词
cases/TEST_CASES.md与test_cases.json	100项同源操作卡，均NOT_RUN

路径	用途
cases/initial_findings.json	16项初始OPEN记录，本地复现NOT_RUN、push未尝试
commands/command_map.json	观察到的真实命令与条件/负例标记，不能无条件批量执行
templates/workflow	仓库、命令、问题、测试、push、CI、候选共七份空模板
evidence/repository_review.json	固定SHA的只读仓库/CI观察，不能作为本机通过记录
evidence/package_validation.json	交付包结构验证结果，只验证本包完整性
tools/validate_package.py	仅校验本包的JSON、ID、状态和链接，不运行OpenBot或Git
reports	本执行手册PDF；详细100项操作卡仍以cases文件为准

E0–E7是实施顺序；G00–G15是最终发布条件。两者编号不一一对应，需由实际候选记录把每个门禁关联到用例与证据。

主文档引用索引

下列T编号供docs/01、02引用。其他章节使用本章内的P/G/S/R局部编号，均附直接一手来源链接。动态官方资料按2026-11-05读取；执行时须核对锁定版本，不能把main文档的新合同强加到旧版本而不做适配。

编号	一手来源	支持的判断
T01	GitHub工作流目录规则	.github/workflows下不支持子目录工作流发现
T02	runner-images退役公告、当前runner标签	macOS14于2026-11-02退出GitHub Actions；实际标签及架构须再验证
T03	pytest导入机制	python -m pytest会额外将当前目录加入sys.path；并非导入问题的普遍免检修复
T04	uv的Python版本管理	可下载Python构建受uv版本与目标平台影响，安装方案需实际验证

此次验证与未验证

已经验证的是本包的用例字段、ID/任务引用、初始状态、JSON可读性、相对文档链接及PDF版面。逐页渲染后检查边界和缺失字形，并抽查关键页面的可读性；具体结果保存在package_validation.json。所有这些检查只涉及交付文档。

尚未执行的是用户Mac上的任一产品用例、真实租户/模型/OS故障场景、任何修复commit与push。没有生成虚构的通过数、测量时长、修复SHA、远端回执或新CI结果。当前仓库审查的历史CI失败保留原事实，新的候选结果须由实际执行产生。